

Sleeping Machines

Deep learning that computes with time

Tero Keski-Valkama and Karoliina Salminen · Research report · 30 September 2026

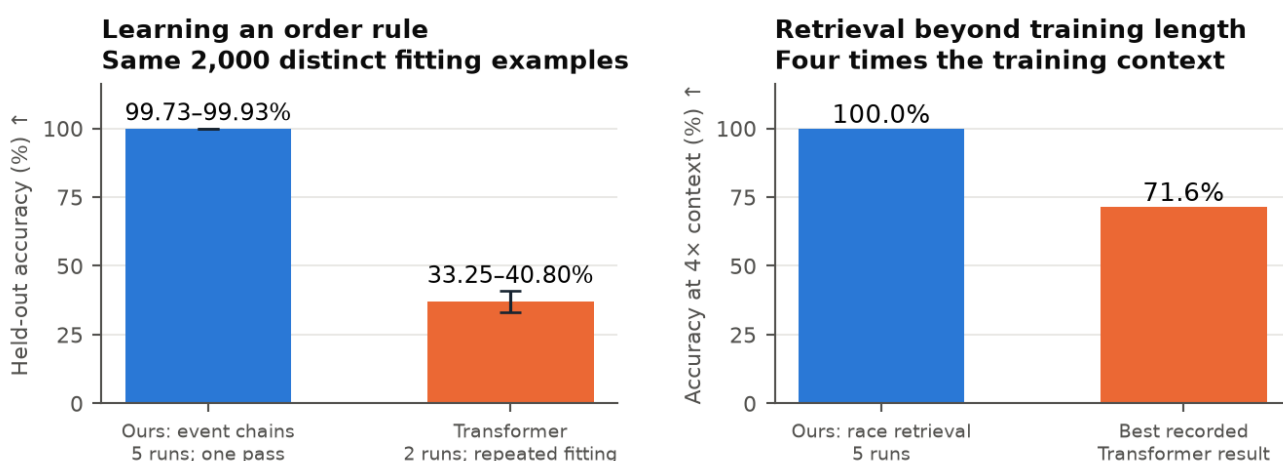
Messages carry content and an arrival time. Nodes mix incoming vectors with persistent memory, gate their updates and compete through learned delays. Arrival order and winning races determine the computation. The goal is useful intelligence with much less active work.

The differentiators at a glance

- **Time performs computation.** Delays, races and phase transformations implement useful functions.
- **Hard routes can learn.** Winning messages execute; unrealized alternatives receive counterfactual credit.
- **Deep, persistent event representations.** Vector messages and local memory carry information and credit through layers; retrieval and temporal primitives share the model family.
- **Capacity beyond activity.** The scaling goal is more useful dormant capacity, selectively recruited and judged by prediction quality at a given total work budget.

The strongest demonstrated results

- **Generalization.** Race retrieval reaches **100% at four times the training context** within 4,000 examples in all five runs. A learned phase rule solves **all 3,440 unseen modular triples**, using the supplied period 17.
- **Learning from fewer examples.** Depth-three event chains reach **99.73–99.93%** after 2,000 examples seen once; saved Transformer controls reach **33.25–40.80%** with the same number of distinct examples and repeated fitting. Depth-four chains reach 99.9–100%.
- **Learned representations.** Completed temporal-carrier development screens reach **2.572 bpc at 131K** and **2.210 at 1M fitting characters**, four passes. A learned speech encoder reaches **79.69%** on 512 private development utterances. Embeddings, temporal state and vector maps learn.



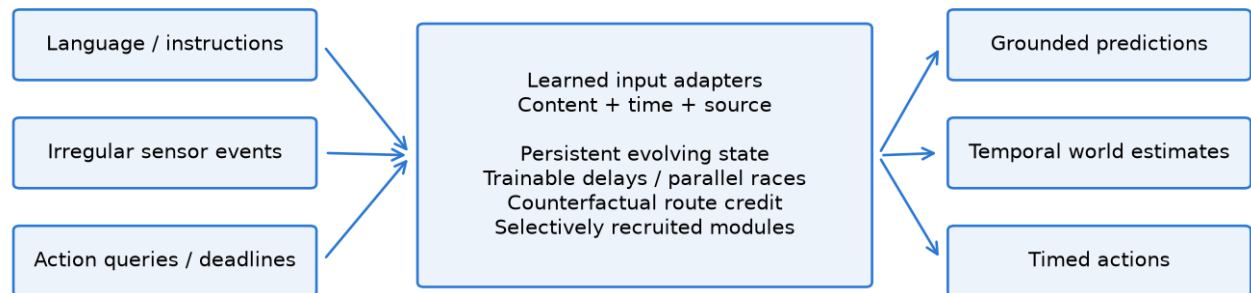
Left: means and recorded ranges, five event runs and two Transformer runs; 2,000 distinct examples, seen once / presented 400,000 times. Right: all five event runs reach 100% within 4,000 examples; the control is the best saved result across seven Transformer configurations and their learning curves. These synthetic tasks use different architectures and structural priors. Sources: E53/E36 and E61.

Language scale-up: the integrated sparse/timed architecture is now prioritized. The comparable 10M-character test remains pending; completed neural controls and costs are in Appendix B.

A general architecture for content, time and selective activity

Language tokens, irregular observations and action requests can be expressed as content-bearing events with timestamps and source identities. Sleeping Machines aim to learn through this common interface: local state evolves between arrivals, delays perform computation, and only recruited modules act. This broader design is the central research target.

Ours = Sleeping Machines



Why this could matter across domains

- **Ordering as computation.** Earlier events change the state/routes encountered by later ones; elapsed time changes that state. The structured order-learning results support this prior for sequence-sensitive signals.
- **Asynchronous sensing.** Updates can follow observations and required deadlines rather than a periodic sweep of all modules. Silence remains informative when the objective depends on waiting time.
- **Instruction-conditioned control.** Language can guide event routing and memory; observations can ground language and update a world state that informs timed actions.
- **Useful dormant capacity.** Stored modules need not all execute for each input. The gain depends on economical discovery and credit, and is judged at a fixed total work budget.
- **Distributed hardware.** Local event-triggered state and communication can reduce global coordination. Globally clockless ASICs are a target; conventional FPGA prototypes retain clocks. Hardware joule savings remain to be measured.

What is established, and what is next

Ours: family evidence	Completed result / scope	Next generality test
Temporal reasoning	99.73–99.93% event-order accuracy; five runs, declared structured task	Unseen delays, gaps and concurrent streams
Deep learned context	3.121 development bpc; sparse six-block / 32K fit; eight-block models also train	Matched-quality work and capacity scaling
Auditory events	79.69% on 512 private development utterances; selected temporal encoder	Aligned official-test real-stream comparison

Joint integration is a research target. Existing results use separately trained task-specific variants; the language candidate has 27 character pools. Shared-weight multimodal learning, camera advantages and robot reliability need dedicated benchmarks. Sparse conditional routing also motivates tabular prediction: paired delays can represent feature thresholds (theory §323). Preserve feature IDs and avoid invented row order; trees and modern tabular Transformers remain controls. No tabular advantage is established (TABULAR_RESEARCH_PROTOCOL.md).

The research upside: five routes to useful advantage

The investment thesis is a trainable substrate with a known useful workload and broader capability beyond it. Reproducing relevant Transformer quality and convergence with lower whole-system energy would already be valuable. Temporal expressivity, selective capacity and cross-modal integration offer additional, independently testable upside.

Strength	Potential benefit	Evidence needed
Temporal races and local state	Less digital normalization and value aggregation; a path to globally clockless hardware	Matched-quality full learning/inference, precision, throughput and measured system joules
Evolving state and reused matches	More useful transformations per expensive match; potentially smaller models	Width/depth/data sweeps at fixed quality and complete work
Capacity beyond activity	More specialized stored representations without executing all modules per observation	Improve quality as capacity grows; keep discovery, teaching and execution budgets economical
Common content/time interface	Language-guided sensing and event-grounded reasoning/control in a shared model	Joint held-out modality combinations, task success and causal deadline tests
Native learning during use	Adapt delays/routes/content locally on an event ASIC; reduce external trainer traffic	CPU online: 3.191→3.096 bpc; asynchronous on-chip credit/updates remain untested

Quantitative scenarios, with conditions

- **Compression.** At equal quality, half the width would give one-quarter projection arithmetic and roughly half the matching/state work. This compression is unproven.
- **Energy scenario.** Assume baseline shares of 40% compute, 40% memory, 10% clock, 10% fixed. Halving compute/memory energy, removing the clock and adding 5% control gives 45% savings (1.82×). These are assumptions, not a chip forecast.
- **Dormant units.** H2 stores 864 receivers and selects 16 updates/character (54× capacity/activity). Teaching evaluates 32 receiver alternatives and admitted historical values; shared maps execute. This is not a 54× resource saving.

Why this is a research program worth testing

Multi-run structured learning/generalization, temporal algebra and deep trainable event representations provide starting evidence. The current language fits have not established a matched-quality resource advantage. Optimizer, width/head/data scaling and repeatability tests address that gap. Theory §320 proposes repeated Poisson arrivals that reuse matched keys without resetting all losing clocks.

Current autograd, global clipping and block-window Adam do not demonstrate fully asynchronous learning. That needs dependency/version-aware credit and tested updates. On-chip learning has precedents (Intel Loihi 2); the proposed contribution is the complete temporal/sparse-credit construction. Compare competent synchronous learning ASICs and charge gradient/optimizer traffic. See `HARDWARE_VALUE_PROPOSITION.md` and `EVENT_STREAM_ADVANTAGE_PROTOCOL.md`.

The hypothesis: more capability per unit of active work

Sleeping Machines combine trainable delays, temporal races, evolving local state and counterfactual credit. The hypothesis is that these mechanisms can approximate useful attention with less selected arithmetic and value movement, then use richer temporal computation and dormant capacity to reach comparable quality with smaller models or less fitting. A common content-and-time event interface can support tokens and irregular sensor streams, with task-specific adapters and losses. Existing cross-task models train separately; the integration target is language-guided event routing and shared state: events ground language and both inform actions. Shared-weight multimodal learning remains a further milestone.

A softmax race samples exactly from its distribution. One winner does not equal its weighted average. Averaging m independent winners has mean-square error $\text{variance}/m$; approximate Transformer containment also requires historical coverage and stable propagation through depth. Temporal state permits additional computations beyond this attention analogue.

Matched attention work: retain all query/key matches

Let d be total width, L depth, N historical keys, r the feed-forward expansion and $B = (8 + 4r)d^2$ the shared projection/content work per layer. S is extra evolving-state work per layer. Two FLOPs per multiply-add:

Per token / target	Transformer	Ours: race substitution
Inference	$L(B + 4Nd)$	$L(B + 2Nd + S)$
Training, approximate	$3LB + 12LNd + 19P/U$	$3L(B + S) + 10LNd + 20P/U$
Logical value reads (FP32)	$4LNd$ bytes	$4Ld$ bytes
Logical key + value reads	$8LNd$ bytes	$4L(N + 1)d$ bytes

P is updated parameter count and U targets per Adam update. The race training term includes admitted losing-value credit; it is not winner-only training. Our normalization of accumulated gradients adds one operation per updated parameter. Shared embeddings/output and lower-order operations are added in the plotted scenario.

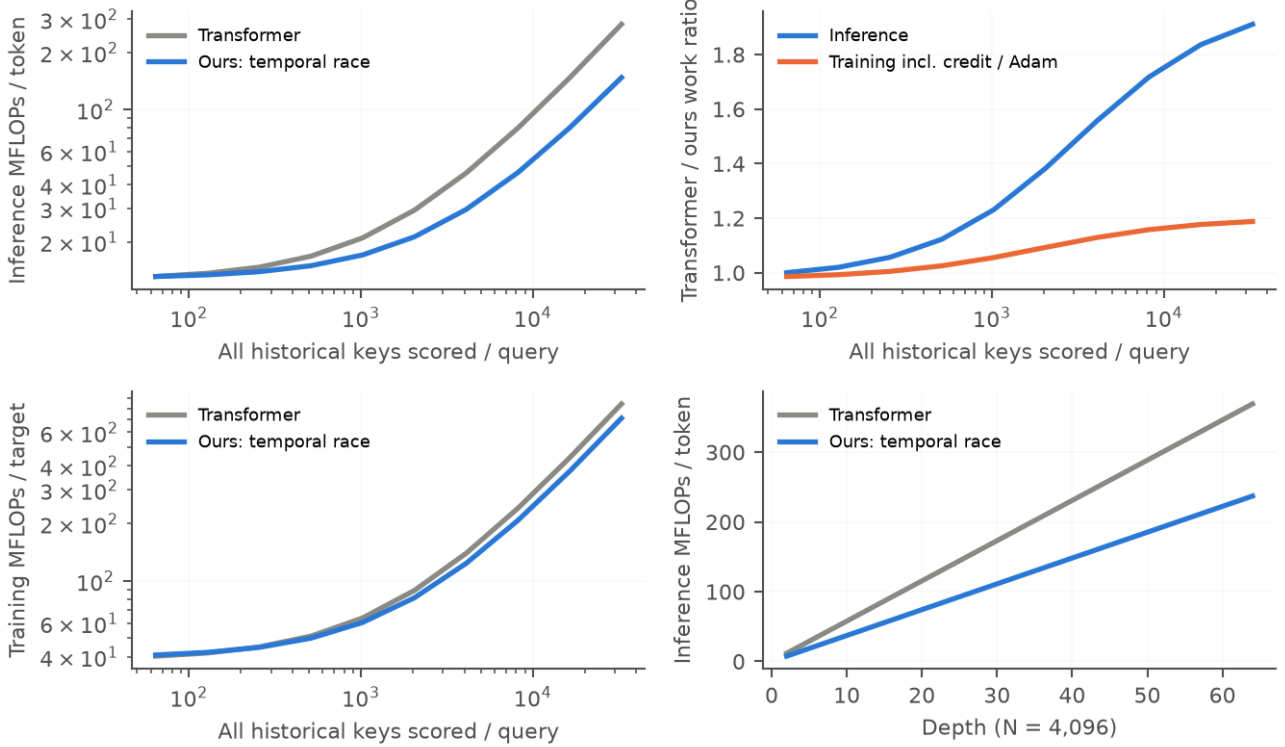
What would make the case decisive?

- **Matched-quality efficiency.** Repeated completed comparisons of full fitting work and inference work.
- **Capacity beyond activity.** More useful stored modules with nearly fixed routing and execution budgets.
- **Temporal expressivity.** Reuse expensive matches for distinct cheap races and evolving-state responses; test whether this reduces required width or depth.
- **Common event interface.** Tokens, irregular sensors and instruction-conditioned control can use content-and-time events. Real-stream, joint-reasoning/control and hardware-energy advantages require their own benchmarks.

This is a research hypothesis and an architectural comparison, not a frontier-language or measured-energy claim. Current deep sparse learning and structured-task results establish meaningful mechanisms; compression and broad language advantage need further evidence.

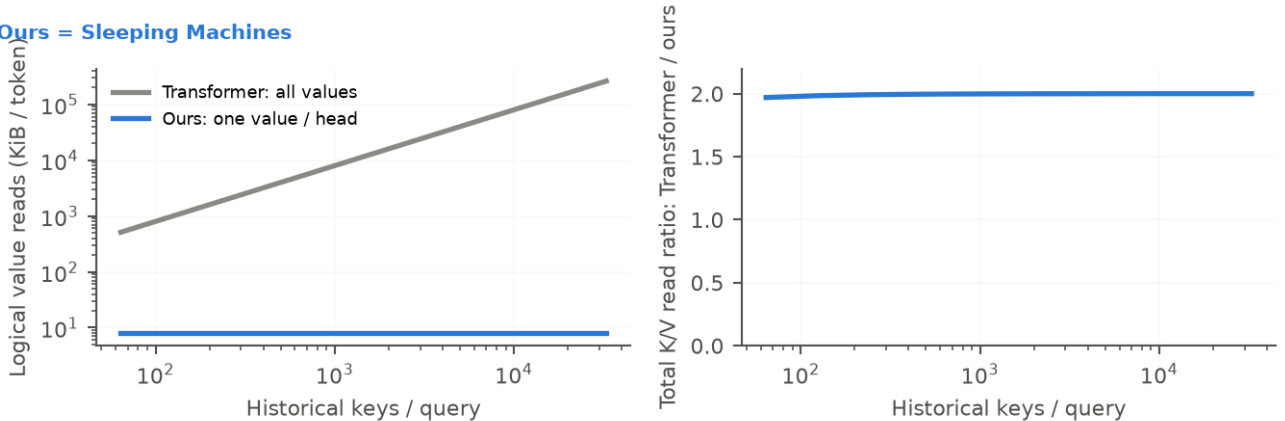
Expected architectural work and access scaling

Ours = Sleeping Machines



Scenario: $d = 256$, four heads, $r = 4$, $U = 128$ and $S = 128d$ per layer. Context plots use $L = 8$; the depth plot scores $N = 4,096$ keys. All keys are scored in both models. Both retain linear context and depth terms, and quadratic width terms. At fixed width, the attention-only arithmetic limit is about $2\times$ at inference and $1.2\times$ during counterfactual training; common projection work lowers these total-work ratios. If richer temporal computation reaches the same quality at width αd and depth βL , projection work scales by $\beta\alpha^2$ and context work by $\beta\alpha$. Those additional savings require matched-quality evidence.

Ours = Sleeping Machines



Winner-only retrieval reduces logical value reads by N in this one-sample scenario. Including the key reads, total K/V access improves by at most about $2\times$. These counts are logical accesses, not measured off-chip transfers, cache behavior or joules. Multiple winners increase value reads. Explicit digital probability normalization is avoided in a physical race, but clock circuitry and rate setting still have costs.

Shared content/projection structure isolates the attention substitution; this is not a quality-matched fit of our current receiver model. Additional receiver-alternative teaching, indexing and scheduling must be charged when present. Bounded candidate search is a separate coverage hypothesis. See theory note 48, §§313–323; measured quality/work curves remain in the appendix.

Why this research matters

Sparse neural computation promises to spend work only where information changes. The hard part is teaching useful deep representations when routes can be silent, discrete or absent. A cheap forward pass is insufficient if discovering those routes consumes the savings.

Research lineage	What it established	The question we pursue
Neuromorphic and spiking networks	Event-driven signals and trained spike timing; EventProp differentiates at events.	How do inactive alternatives receive useful credit without exhaustive replay?
Temporal logic and learned delays	Race/delay algebra computes with time; delay learning already improves SNN recognition.	Can temporal computation coexist with rich vector content and deep learned state?
Sparse conditional models / MoE	Selected experts allow capacity to grow faster than active work; Switch trains at scale.	Can message timing, communication and correction work also become selective?
Asynchronous state-space models	EventSSM learns asynchronous streams with parallel scans; this is a strong precedent.	Can hard races and counterfactual alternatives add quality per unit of total work?

What is distinctive here

We combine computation through trainable time, content-bearing messages, persistent local state and credit to unrealized alternatives. Optionality asks whether distinct, reachable future corrections remain available under a work budget. The theory connects temporal algebra, key/value separation, credit transport and supervision that includes silence. The contribution is this construction and its tested consequences; learned delays and sparse capacity are established ideas.

The integrated language candidates now exercise learned content, temporal races, sparse persistent receivers and counterfactual credit together. An eight-block variant also races historical key/value messages. These small-data experiments explore only a small part of the design space; larger-scale quality and complete resource advantages remain under test.

Our earlier deep sparse-routing pilots often lost activity and useful credit before the final layers. Counterfactual proposals alone did not reliably fix that. The subsequent vector-state and persistent-memory work addresses those observed obstacles. Completed structured-task gains motivate the larger learned-model tests; broad quality, training efficiency and energy must still be measured together.

Primary precedents: EventProp; Space-Time Algebra; Learning Delays in SNNs; Switch Transformers; EventSSM. Our routing failures and revised interpretations remain in the theory index and findings.

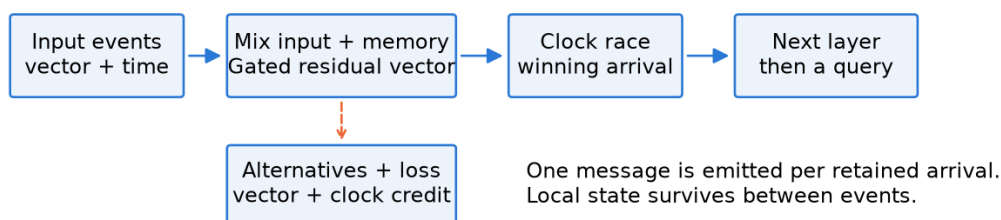
One architecture, several learned computations

The common idea is local computation triggered by an arrival: retain memory, combine the incoming vector with that memory, and choose an outgoing time. A delay changes which messages meet and which race finishes first. This makes timing part of the learned function. The model family implements this idea at several levels of generality.

Ours = Sleeping Machines

Ours: a learned event-state block

Source identity, content and elapsed time determine the next vector and arrival.



Model	Mechanism	Evidence	What it establishes
Ours: integrated sparse temporal language	Learned content, state-dependent key races, selected receiver updates and episodic KV	Completed 32K receiver / 2K depth and KV screens	Combined mechanisms train; bounded candidate coverage and causal schedule
Ours: learned event-state encoders	Source embeddings, temporal modes, nonlinear vector maps and competing clocks	Language E176; speech E165	Learned representations and persistent state
Ours: routed event query encoders	Candidate payloads, receiver memory and hard value/time races	Breadth E120; language E133	Trainable event depth across tasks; text/market breadth variants add statistical evidence
Ours: structured event mechanisms	Temporal chains, relative pointers and phase composition	E34/E53/E54, E61, E124	Sample efficiency and generalization with declared structural priors
Ours: statistical controls	Conditional counts, backoff and copy probabilities	E173; online pilot	Separate baselines; no event backbone

Incoming content is retained. In the event-state encoder, an incoming vector is projected into rotating, decaying memory. A learned memory read is mixed with a direct input path, normalized and gated. The outgoing vector adds that correction to the incoming vector. The payload therefore depends on both current content and history; timing supplies an additional control.

Each task has separately fitted weights. Current learned encoders use fixed depth and locally dense vector maps; the language scheduler retains state and delayed messages across chunks. The routed query encoder instead rebuilds a supplied context per query. Clock learning uses declared surrogate credit through a hard schedule; it is not an exact derivative of every order change.

How the model computes and learns

Time performs computation. An arrival time is a computational value. A delay adds to that value; a first-arrival race computes a minimum and selects a payload; coincidence detects the latest required arrival. Inhibition can veto a path. With a declared periodic reference, phase transformations compose reusable modular relations. Learned timing therefore changes the function and its causal paths, beyond deciding when a fixed dense calculation runs.

- **Local temporal memory** accumulates observed content and elapsed time without evaluating empty time ticks.
- **Computation through delays** uses waiting times, arrival order and clock races to transform information and select outcomes.
- **Hard races and counterfactual credit** choose the emitted vector and delay. Losing alternatives teach better routes while remaining distinct from the winning forward message.
- **Separate keys and values** let content-dependent keys set routes and clocks while value learning preserves the selected schedule.
- **Trainable depth** carries representations and learning credit through a hierarchy. The reversible construction preserves conditional norms under its stated assumptions; readout visibility and routing remain necessary.
- **Structured memories** include relative pointers, learned phase transformations and conditional evidence. Their task gains retain their declared priors; statistical experts are labeled separately.
- **Appropriate supervision** teaches a completed class decision or the next event's type and waiting time, including information carried by silence.

For a temporal pattern such as A followed by B, a learned delay can bring A's trace into coincidence with B. A competing path can veto the match when C intervenes. The timing-pattern and compositional experiments test these mechanisms; the language and speech encoders learn richer vector messages and temporal state.

The primitives and their symmetry limits are developed in temporal computation theory, §56; counterfactual learning, key/value separation and reversible depth give the learning contracts. Clockless delay/race networks compute relative timing relations; phase arithmetic requires its reference. Each implemented model uses a declared subset. Candidate discovery and training alternatives are charged to the work ledger.

The ambition: useful capacity without proportional activity

The proposed shift is to compute through event timing and selectively active paths. A larger network should be able to retain more useful dormant structure while spending work on the paths a query needs. Counterfactual credit must teach those hard choices, including useful alternatives that did not win. The earlier temporal-chain, pointer and phase results test parts of this case and remain central evidence.

What temporal softmax actually provides

If candidate clocks have rates $\exp(\text{score})$, their first-arrival winner has exactly the softmax choice probabilities. Competition supplies normalization in time. It can avoid an explicit normalizing sum/division in the winner path when those rates are physically available. Score formation, candidate discovery, value delivery and learning still cost work. This identity is not a measured near-zero-energy attention system.

Ours: mechanism	Completed evidence or status	What remains
Temporal chains / hard pointers / phase rules	Strong structured-task accuracy, transfer and work comparisons	Transfer the useful priors to broad learned representations
Temporal content carrier	Learned embeddings, state, gates and delay-dependent transport	Every layer executes; does not demonstrate dormant-unit scaling
Integrated sparse temporal model	Hard races, content memory, sparse state updates and counterfactual teachers pass contracts	Scaling quality and complete learning work are under test
Capacity beyond activity	Gains depend on useful sparsity and learning	Measure marginal useful capacity with bounded active work

A direct mechanism experiment

The prioritized model combines the mechanisms: a character event enters six timed races, selecting one persistent content-bearing unit at each depth. Separate state-dependent keys set rates; the winner mixes incoming content and retained memory, then emits a vector and learned arrival time. Addressed losing values receive counterfactual score credit during training. The dense carrier and carrier-plus-retrieval variants remain diagnostic controls.

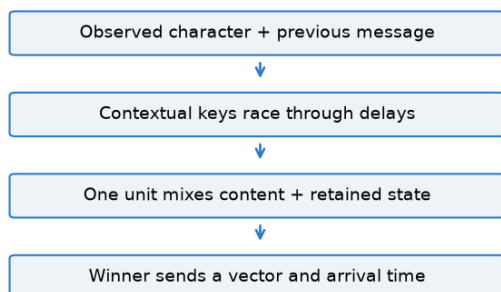
Character-indexed pools and fixed depth are declared priors; learned topology growth and unrestricted asynchronous schedules remain open. RNG and physical traffic are additional. The old time-normalized value sum has a shared random amplitude; its covariance and cutoff claims are corrected beside the original theory, not silently deleted. A centered, conserved teacher is now tested. Its fixed-error expected Jacobian is not an unbiased sampled-loss gradient. Theory §§294–298: [experiments/theory/45_race_attention_and_resource_identity.md](#).

Ours: the integrated sparse temporal language experiment

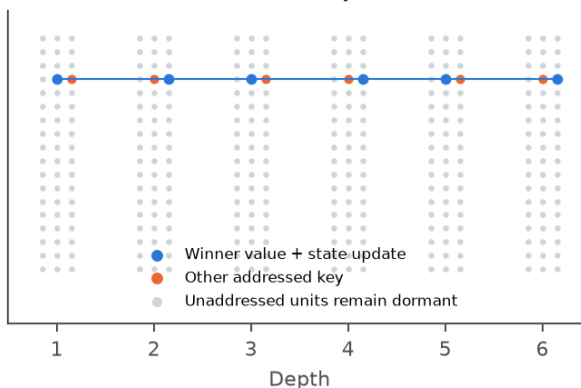
This candidate has no dense language carrier. Each event mixes its embedding with the previous deep message and traverses contextual key races. Only selected receivers update their persistent rotating/decaying state and emit values. Time is part of the computation; inactive receivers are not evaluated on empty ticks.

Ours = Sleeping Machines

Ours: integrated event path



324 available units; 6 state updates / character



The first six-depth, 16-dimensional candidate provides 324 units but updates only six states per character. Each depth scores two addressed keys; training additionally evaluates both candidate values to teach hard choices. Unaddressed pools remain dormant. The capacity experiment doubles available units to 648 while retaining six selected state updates. Key scoring and counterfactual work still grow and are charged.

Checks establish causal predictions, identical chunked execution, precise clocks at 10M positions, equality of training forward values and winner-only inference, learned key/value/memory gradients and conserved route credit. The smoke fit learns, but is not a quality benchmark. Completed data, depth and memory interventions test progress before larger promotion.

A matched payload-32 / 2K depth screen improves 3.633 to 3.542 development bpc from six to eight receiver blocks, for 32.6% more fitting work. The current eight-block KV variant adds historical races, giving sixteen selection steps after warmup. Fixed data/passes/seed do not isolate depth from increased capacity; this is exploratory evidence, not a scaling law.

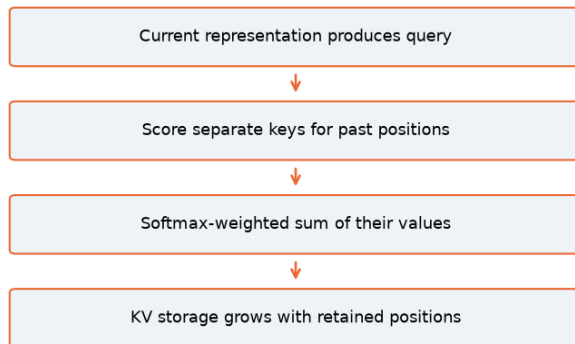
Fixed observed-character pools, bounded delays and six sequential event depths; no learned topology or complete frontier-language claim. Interior arrival-time derivatives and counterfactual score surrogates have distinct scope. FLOPs include teaching alternatives and optimizer work; representative sparse traces do not certify whole-run instruction or energy counts. Theory §§299-302; source: [sleeping_machines/sparse_race_language.py](#).

Ours: queries, memory and context

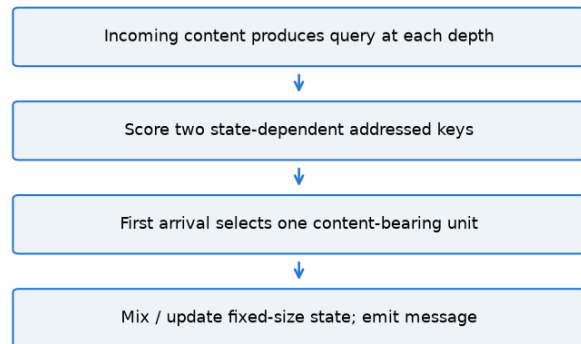
Race attention specifies how a candidate wins; memory organization specifies what the candidates represent. The integrated language model races between compressed persistent receivers. It does not replace a Transformer token KV cache entry for entry.

Ours = Sleeping Machines

Transformer: attention over token entries



Ours: race over persistent memory units



At each depth, a learned map turns the incoming message into a query. A candidate key combines its learned prototype and a read of its retained state. Query-key compatibility sets an exponential clock rate; the first arrival wins with the corresponding softmax probability. The observed character addresses two candidates per depth. Queries cannot search arbitrary past-token keys in this construction. Winner-only delivery also differs from a deterministic softmax-weighted sum, although its one-step expectation equals that sum.

Model / storage formula	Raw state / one stream	Forward history
Ours: width 16, 324 states + clocks + last message	22.84 KiB	Carried until stream reset
Ours: width 32, 324 states + clocks + last message	43.16 KiB	Carried until stream reset
Transformer: conceptual FP32 KV, 4 layers × 256 positions × width 256	2,048 KiB	At most 256 characters

The raw width-32 state is about 47× smaller than this conceptual KV allocation and does not grow with history length. This is a storage-formula comparison, not matched recall capacity or measured total RAM: ours compresses history, whereas KV entries retain separate position-addressable representations. The saved Transformer actually recomputes windows without an implemented KV cache. Its 256-character window is a model setting, not an intrinsic dataset limit. Ours retains forward state beyond its 16-character training-credit horizon.

Ours raw bytes = $324 \times (4d + 8) + 4d$; conceptual Transformer KV bytes = $2 \times 4 \times 256 \times 256 \times 4$. Excludes weights, gradients, optimizer, activations, object/index overhead and traffic. Long-range recall and comparable-quality memory advantages remain to be measured. Recurrent compression resembles the memory organization of selective state-space models (Mamba, Gu & Dao, arXiv:2312.00752); our hard temporal races and counterfactual route teacher are separate mechanisms. Compression is not required by races: a separate integrated per-position KV experiment retains historical entries and tests sparse value delivery. Theory §§308–310.

Ours: completed integrated-language stages

Ours: fit / payload / pool	Development bpc ↓	Fitting GFLOPs ↓	Capacity / selected states
32,768 / 16 / 2	3.121	29.88	324 / 6
8,192 / 16 / 2	3.398	7.49	324 / 6
8,192 / 16 / 4	3.426	14.15	648 / 6

These are the integrated model stages, with identical cold development targets. Each character selects one unit at each depth; addressed alternatives teach the races. Capacity and selected activity are different counts. The fitting ledger includes counterfactual values, backward, clipping and Adam.

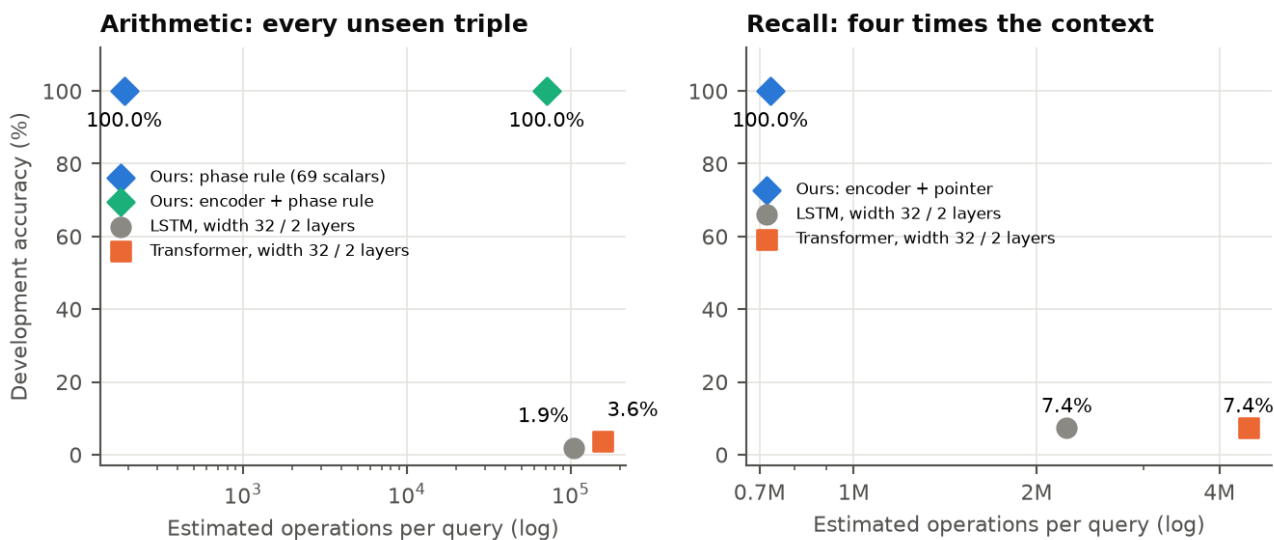
Quality, data efficiency and work must be judged together. Completed earlier carrier results remain preserved. Comparing these models also changes payload size, capacity and truncated credit, so a score difference does not isolate one mechanism. No official test or energy measurement is implied.

One seed; observed-character index; no statistical expert. Whole-stream candidate scores, selected updates and teaching visits are recorded. Arithmetic is a representative saved-parameter extrapolation; sparse optimizer activity depends on the actual input and credit history. RNG, indexing and memory traffic are separate. Source: `experiments/results/parallel_language/local_full_sparse_language_*.json`.

A demonstrated advantage: generalization with less work

The retrieval and phase computations preserve useful rules when the evaluation extends beyond the fitting examples. The saved compact Transformer and LSTM controls use the same synthetic evaluation targets. **Higher and further left is better:** more accurate answers from less estimated inference work. All points use one logical operation ledger.

Ours = Sleeping Machines



Ours: event computation	Held-out capability	Estimated work per query
Periodic path; 69 learned scalars	3,440/3,440 unseen triples	188 logical operations
Two-layer carrier + hard pointer	100% at four times context	728,602 logical operations

The phase rule costs 188 logical operations per triple; the saved LSTM and Transformer cost 104,518 and 155,592 and score 1.95% and 3.60%. At four times the recall context, the event encoder plus learned pointer scores 100% at 728,602 operations; both compact controls score 7.42% at 2.24M and 4.46M operations. The phase model receives a periodic representation with period 17, and retrieval has a pointer mechanism. These useful priors explain the task advantage and are part of what must transfer to harder tasks.

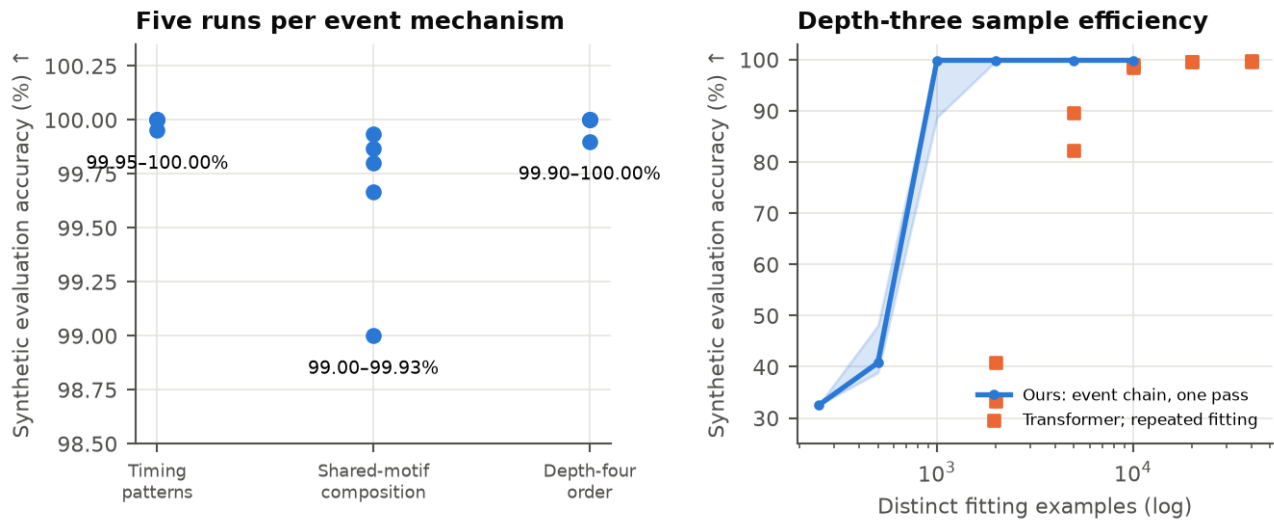
Learning work is also selective. The periodic teacher makes 29,003 mistaken-example updates and 145,015 learned-scalar update visits. The dense arithmetic controls make 4,800 Adam steps: 92.2M parameter visits for the LSTM and 132.5M for the Transformer. These count parameter updates, excluding optimizer state and backward arithmetic; they are not training FLOPs or joules.

Arithmetic: 1,473 fitting triples, a 200-epoch budget, all 3,440 unseen triples; supplied period 17. The phase-only path stops after 47 passes, when an entire fitting pass makes no updates. Recall: 4,000 pointer-fitting examples plus 512 neural-fitting examples; the dense controls receive all 4,512 examples for eight epochs. Width 32 and two generic layers were present, seed 6, one small dense setting. Work is an analytic logical-operation estimate, including configured vector maps, routers, scans, normalization, clock candidates and pointer search. These inference counts are not measured joules or backward/optimizer counts. Full work definitions appear in the evidence appendix.

A demonstrated advantage: learning temporal structure

Event chains learn timing patterns and compose recognizable parts into ordered structures. The preserved five-run results show accurate recognition and strong sample efficiency. The right panel compares distinct examples rather than incompatible activity counters.

Ours = Sleeping Machines



Preserved comparison	Ours	Transformer	Fitting protocol
Timing patterns	99.95–100%; five runs	99.60–99.80%; two runs	200k examples once / 1M with relative-time bias
Shared-motif composition	99.00–99.93%; five runs	99.60–99.85%; two runs	40k examples once / 1M with relative-time bias
Depth-four order	99.90–100%; five runs	98.95–99.05%; two runs	Same 40k distinct examples; one pass / 50 passes
Depth-three order at 2k examples	99.73–99.93%; five runs	33.25–40.80%; two runs	One pass / repeated fitting on the same 2k examples

The strongest depth-four comparison has approximately ten times fewer classification errors despite the event learner seeing each fitting example once. The sample-efficiency curve makes the next question concrete: can learned embeddings and broader event representations retain that advantage when the inputs no longer supply known temporal parts?

Completed E35, E34, E53/E54 and E36 files. Ranges describe the recorded runs, not confidence intervals. Architectures and optimization differ; synthetic evaluation sets were reused during research. Event deliveries remain activity measurements. Total arithmetic, backward and optimizer work require a declared ledger; activity divided by dense MACs is not a training-cost or power ratio.

Why asynchronous, sparse computation matters

Persistent local state can retain experience without repeatedly reconstructing a whole history. An asynchronous node updates when useful information arrives. A hard race emits one chosen message. These mechanisms create a path to spending less computation and moving less data per useful answer. On an event-oriented processor, inactive nodes and communication links could remain idle.

Spend the next unit of work where it helps

A larger budget can buy a longer memory, better retrieval, a deeper representation for difficult inputs or more local adaptation. The development rule is to measure the held-out improvement from each choice and allocate work to the most useful one. The phase and pointer results show why an appropriate computation can be much cheaper than a generic dense approximation. The scaling program tests how much of this flexibility survives when that computation must itself be learned.

Architecture	Fitting known sequences	Generating or processing a stream
LSTM	Gates depend on prior hidden state; recurrent work is sequential	Retained hidden state; dense gate updates per token
Transformer	Causal attention permits sequence-parallel fitting	Sequential token generation with a key/value cache; adaptation is possible
Ours: Sleeping Machines	Affine event memory supports parallel scans once incoming values/times are known	Retained local state and delayed-message queue; only due arrivals execute

Parallel fitting and sequential generation are compatible. The precise-clock language scan gives a 12.30× complete-step CPU speedup against serial execution of the same width-256 model, including backward, clipping and Adam. Predictions, states, gradients, chunk boundaries and causality are checked. This follows a bounded-delay schedule; arbitrary reordering networks require their own contract.

Parallel recurrent computation also appears in linear attention and selective state-space models. EventSSM already processes asynchronous events with scans. These are important controls. The distinctive hypothesis here is the combination of learned timing, sparse communication, credit to alternatives and independent compute budgets; it must earn its advantage empirically.

Primary FLOPs describe the declared event algorithm, including required vector maps, candidate computation, scans, backward and optimizer updates. Simulator padding and dispatch are separate implementation overhead. Physical power also depends on memory, queues, communication and hardware utilization. Demonstrated work savings and projected power savings are distinguished; total device joules have not yet been measured.

A mathematical foundation for trainable computation

Principle	What it enables
Stable transport through depth	The reversible packet/memory construction preserves conditional value and credit norms under its stated operator and boundary assumptions. Readout visibility, routing and optimization remain separate requirements.
Active communication support	Inputs need causal paths through which to interact. A context channel supplies joint information when sparse packets leave local groups disconnected.
Credit to unrealized alternatives	A losing payload or timing choice can show how a different route would change the outcome, while forward computation remains a hard race.
Periodic state as an isometry	Learned rotations/reflections have unit-magnitude occurrence derivatives. Their composition supports reusable arithmetic instead of a table of observed tuples.
Certified composition	Target-constrained min/max composition of phase errors certifies the fitted modular rule across all 4,913 possible tuples; exhaustive checking confirms it.
Natural supervised credit	Categorical and event likelihoods both credit predicted sufficient statistics minus observations. Silence enters through integrated exposure.
Useful optionality	Reserve consists of distinct, attainable future corrections under a causal work budget. Reachability and transferable learning matter alongside immediate loss.
Statistically useful credit	Expected improvement must overcome the curvature cost of fitting noise. Cross-example teacher agreement separates reproducible correction from raw gradient magnitude.

From mathematics to an engineering discipline

The theory connects representation, topology, clocks and optimization. Expressivity describes what a network can compute; transport describes whether information and credit survive; the objective describes what the teacher asks it to learn. These pieces must agree. The periodic certificate is one concrete case where the formal model explains and verifies a learned computation.

A common implementation makes these principles reusable across tasks. Efficient primitives can own a computation when its structure is known; a deep carrier can learn representations when it is not. The research objective is to combine this flexibility with affordable route discovery and increasingly capable models.

Formal derivations and their assumptions are indexed in the project's theory notes. Conditional stability and a certificate for a fitted rule do not establish global optimizer convergence or a scaling law. The program builds on established deep spiking and sparse conditional computation; its focus is the combination of useful temporal operators, hard causal routing and counterfactual learning. Related survey: Direct training of deep spiking networks.

Potential grounded in the completed evidence

The objective is capable models that spend computation where it improves an answer. The saved structured-task results and learned stream pilots provide specific starting points. Moving from those mechanisms to frontier prediction requires useful representations, longer context and measured quality at a fixed total resource budget.

Opportunity	Present foundation	What would establish the larger case
Compact prediction and memory	Persistent learned state; pointers generalize to longer contexts.	Competitive held-out language quality, retention and complete fitting/inference cost.
Continuous perception	Learned temporal speech representations and composition.	Full speech/vision tests and confidence-based early decisions at measured latency.
More useful training per budget	Structured-task sample efficiency; exact causal parallel scans.	Quality improvements at equal total fitting work, including route discovery.
Dormant skills and selective depth	Hard race routing and compute-allocation theory.	Learned marginal work allocation that outperforms fixed allocation on real tasks.
Mobile and industrial autonomy	Local state and event-triggered updates.	Device joules, memory traffic and task quality measured on deployable implementations.

Why the hardware and economic implications could be large

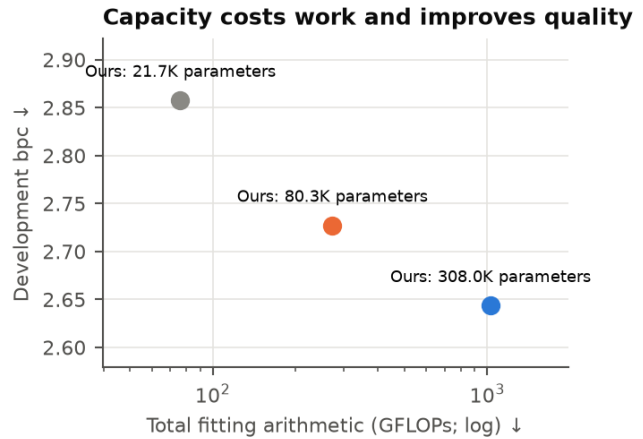
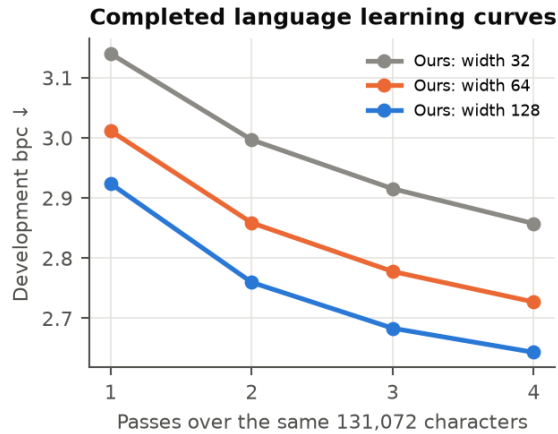
If comparable quality needs less total training and inference energy, a fixed power and capital budget can support more capable models, more research or continuous adaptation on robots, phones and instruments. Persistent local state and selective communication would favor hardware that handles message delivery, queues and memory efficiently. These are conditional consequences of measured savings, rather than savings inferred from event counts.

The near-term experiment asks where the next unit of computation helps: longer memory, richer content transformations, retrieval, depth or local learning. Held-out gains and complete work determine promotion. A successful scaling result must show that those gains continue across independently fitted sizes and budgets.

Ours: language learning and staged scale-up

These learned models use character embeddings, gated residual content transformations and persistent rotating/decaying state. Temporal modes encode relative token distance. No statistical count, copy or word experts provide their predictions. The goal is competitive quality from a learned backbone before claiming a language compute advantage.

Ours = Sleeping Machines



Ours: width	Parameters	Development bpc ↓	Total fitting GFLOPs ↓
32	21,741	2.858	76.11
64	80,301	2.727	272.30
128	308,013	2.643	1026.18

All curves use six layers, seed 6, 131,072 fitting characters, four passes and the same 8,191 cold-context development targets. Credit is truncated every 64 characters; memory persists. FLOPs include forward/loss, backward, clipping and Adam; special functions are reported separately in each result. These runs vary capacity at equal data/passes, rather than equal compute, and establish neither a scaling law nor official-test superiority.

From longer memory to selective content

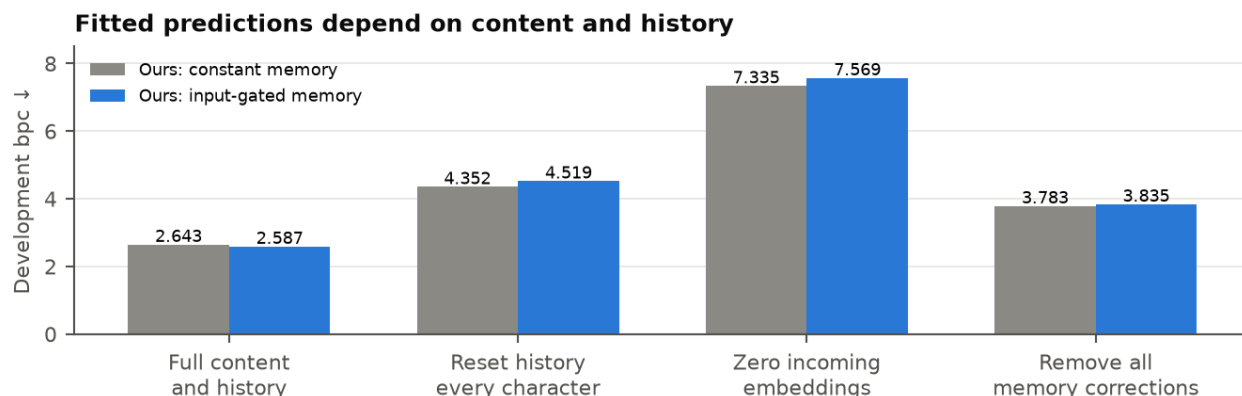
Ours: memory variant	Parameters	Development bpc ↓	Fitting GFLOPs ↓
Constant / inherited	308,013	2.643	1,026.18
Constant / long decay	308,013	2.752	1,026.18
Constant / long spectrum	308,013	2.858	1,026.18
Input gates / inherited	309,561	2.587	1,040.61
Input gates / long decay	309,561	2.627	1,040.61

Identical width, seed, data, four passes and 64-character credit horizon. Constant-memory arms vary initial timescales/frequencies. Input-gated arms add content-dependent write/forget controls (0.50% more parameters, 1.41% more fitting arithmetic). Longer decay alone worsens this fit. These are single-seed development comparisons; complete numerical contracts precede training.

Ours: content and memory in fitted language models

An event carries information about its input. The learned vector is a transformation of incoming content and persistent state, with a residual path and an output gate. The new candidate also gates memory writing and forgetting from incoming content. These checks measure whether the fitted predictions use those paths.

Ours = *Sleeping Machines*



Resetting all history before each character preserves its current embedding and learned content transformations, but raises the gated model's development loss from 2.587 to 4.519 bpc. Zeroing incoming embeddings raises it to 7.569. Both present input and earlier messages contribute to prediction.

Selective retention adds a useful control

The two input gates start at one, preserving the constant-memory model exactly at initialization. During fitting they learn different write strengths and forgetting factors for different incoming vectors. A factor below one slows decay; above one accelerates it. The controls are known from the causal previous layer, so serial execution and parallel affine scans retain their checked outputs and teachers.

In the matched small fit, gates improve 2.643 to 2.587 bpc for 0.50% more parameters and 1.41% more fitting arithmetic. This is a local quality/work improvement, with one seed. All six layers still execute for every character; dormant-unit scaling remains a separate target.

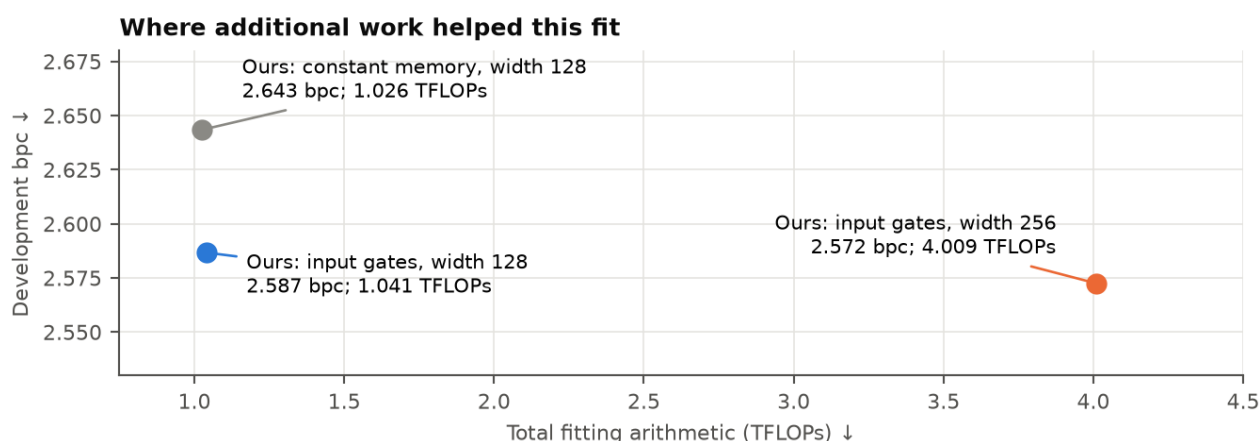
Frozen selected checkpoints, identical 8,191 cold development targets, no training or official-test reads. These interventions disrupt a trained model; they establish fitted dependence, not the quality of retrained ablated architectures or lossless storage. Source: `parallel_language/local_language_representation_20260930T162337Z.json`.

Ours: larger language development stages

Each stage fits independently from initialization. Data, capacity and memory controls are declared below. Development selects weights within the fixed four-pass budget; ongoing training logs are never substituted for a completed result.

Ours: memory / width	Fit characters	Parameters	Development bpc ↓	Fitting TFLOPs ↓
Input gates / 256	131,072	1,208,889	2.572	4.009
Input gates / 128	1,048,576	309,561	2.210	8.325

Ours = Sleeping Machines



On the identical 131K-character/four-pass screen, widening the gated model buys 0.014 bpc for 3.85× the total fitting arithmetic. Adding input gates at width 128 instead improves 0.057 bpc for 1.41% more arithmetic. This makes the allocation question quantitative: measure useful correction before spending broadly on width. These are finite, single-seed interventions, rather than a scaling law.

The fixed-capacity data comparison and fixed-data capacity comparison answer different questions. Equal passes and data do not imply equal compute. The pipeline checks finite learning, trained value blocks, complete work and source provenance before promotion. A development gain is not an official-test or frontier claim.

Precise clocks; float32 payloads; causal persistent state; 64-character credit horizon. Special functions, evaluation passes and physical traffic are separate from the arithmetic ledger. One seed, no statistical experts. Source: [experiments/results/parallel_language](#).

What establishes the larger advantage

The ambition is a common model family whose strongest mechanisms remain useful as tasks, data and capacity grow. Arithmetic and retrieval retain their demonstrated strengths in the consolidated implementation. The strongest language mixture is specialized; the generic backbone still needs to demonstrate competitive learned representations. Character and subword-token budgets must be distinguished.

Objective	Decisive evidence
Generic language scaling	Train a learned event backbone that owns the prediction. Scale through declared data budgets with matched Transformer, recurrent and state-space references; record loss, capacity, complete training work, memory traffic, time and joules.
Preserve capabilities	Repeat established generalization and sample-efficiency results within the common model family, with task-appropriate depth and explicit resource accounting.
Strong real-event recognition	Accurate speech and event-camera decisions on complete held-out benchmarks; calibrated confidence and time-to-answer.
Learn routes and representations at scale	Reliable deep credit and useful counterfactual alternatives as width, depth, memory and data increase.
Efficient persistent operation	Maintain local state and pending messages across queries, preserving causal predictions while reducing repeated work.
Lower total energy at useful quality	Measure training and inference joules, memory traffic, latency and communication under declared hardware and quality targets.

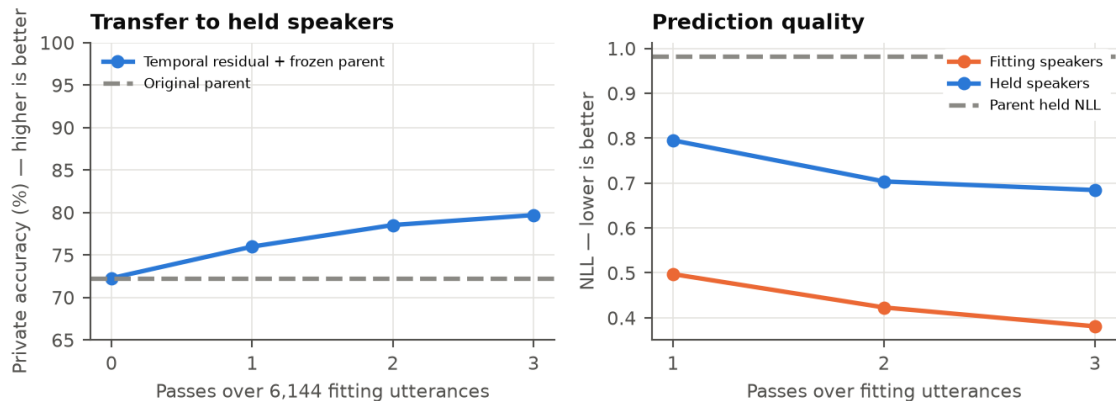
Success on these dimensions would turn the current task-level advantages into a broader foundation for frontier models. The project's distinctive resources—timing, local memory, hard selection and credit to alternatives—remain the guide for architecture and learning.

Appendix A. Deep event recognition

The strongest completed speech result in the model family is **79.69%** on 512 private held-speaker utterances (selected single six-block prefix; trained with twelve blocks). Published official-test results below use a different partition; they are reference targets.

Ours: private development configuration	Correct	Accuracy ↑
Original eight-layer parent	370/512	72.27%
Parent + six-block residual; pass 3	408/512	79.69%
Single six-block temporal encoder; pass 0	406/512	79.30%
Calibrated six-block continuation	400/512	78.12%
Bounded twelve-block encoder	400/512	78.12%
Directional twelve-block encoder	401/512	78.32%
Selected single six-block prefix; trained with twelve blocks	408/512	79.69%

Ours = Sleeping Machines



A six-block width-128 temporal encoder learns corrections while the inherited eight-layer parent stays frozen. It adds 395,814 parameters to the parent's 53,296. Signed modal states, nonlinear gates and residual vectors learn from all source identities and original event times before causal pooling. This is a larger parallel model, not fourteen sequential layers; completed-utterance supervision does not yet teach calibrated early answers. On 657 disjoint utterances from the same held speakers, accuracy improves from 70.47% to 77.63%.

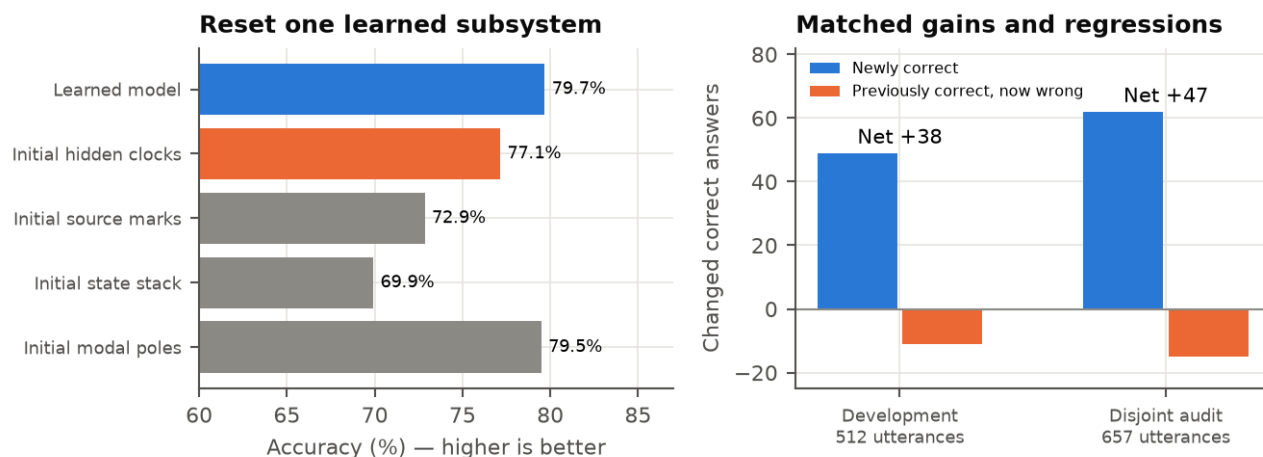
Published reference; official-test protocol	Reported accuracy ↑
Ours: full official SHD comparison	Pending
EventSSM: asynchronous learned state-space layers	95.9%
S7: input-dependent temporal state	96.3%

Our private sample uses training-file speakers 3/6; official test accuracy is unmeasured. The temporal residual uses three passes and a fresh optimizer; its larger capacity and budget are not a matched single-factor comparison. Its listed score selects the best private-development epoch; the curve shows all three. The original parent was fitted on 4,096 examples. Continuation timers exclude loading; the residual timer includes it. Both include evaluation and are not energy measurements. One seed. References: EventSSM, S7. These published scores were checked against the original papers; they are targets for a full official-test comparison, not scores on our private split.

Appendix A (continued). Learned timing and transfer

Learned delays contribute to the answer. Restoring the hidden clocks to their initial values, with source vectors, state/value maps and the trained classifier retained, loses 13 correct answers. Timing changes the temporal interactions used by the representation; it performs computation.

Ours = Sleeping Machines



Ours: same trained readout; subsystem reset	Correct	Accuracy ↑
All learned parameters retained	408/512	79.69%
Only hidden clocks reset	395/512	77.15%
Only source vectors reset	373/512	72.85%
Only state stack reset	358/512	69.92%
Only decay/frequency parameters reset	407/512	79.49%

The temporal stack and source vectors also learn useful coordinated representations. Resetting the stack loses 50 correct answers; resetting sources loses 35. Decay/frequency resets change one decision. These changes depend on the fitted solution's coordination; their effects cannot be added or treated as a matched comparison of retrained architectures.

The 657-utterance audit is disjoint from fitting and the development sample and uses the selected checkpoint unchanged. It gains 62 correct answers and loses 15, for a net improvement of 47. Both samples use the same two held training speakers. Official-test and additional-speaker generalization are the next evaluation targets.

A stronger mathematical account of routing

An affine packet summary can preserve both the final state and the average of raw temporal states, including their teachers. This permits richer pooling before expensive nonlinear maps. A second derivation shows why many almost-equal delays may offer little usable choice: their effects point in nearly the same direction. Diverse payloads and temporal modes, sufficient delay spread and downstream visibility determine useful route reserve.

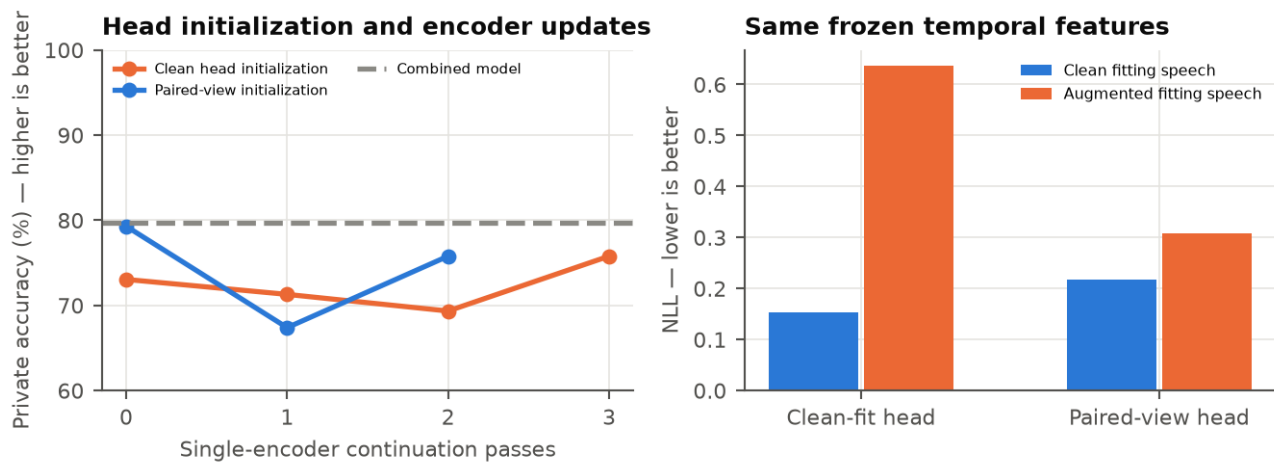
One exploratory run: 449,110 total parameters, inherited parent plus a trained six-block encoder. Three passes use about 33 minutes including preparation/evaluation and 1.79 GiB peak RSS; the guarded host retains at least 10,361 MiB sampled available memory. Complete physical work and joules remain unmeasured. Formal statements and numerical contracts are in THEORY §§226–236.

Appendix A (continued). One temporal encoder

A six-block temporal encoder retains nearly all the combined model's development accuracy through one ordinary query head. Deployment removes the frozen parent: 395,814 parameters replace 449,110. Modal states, gated vector messages and winning delays remain. Its weights inherit earlier encoder training; combined teacher predictions are used only to initialize the head.

Ours: private development configuration	Correct	Accuracy ↑	NLL ↓
Combined model: parent + temporal correction	408/512	79.69%	0.684
Single encoder: clean head, before continuation	374/512	73.05%	0.818
Single encoder: paired head, selected pass zero	406/512	79.30%	0.631
Single encoder: selected trained prefix	408/512	79.69%	0.625

Ours = Sleeping Machines



Fitting the head on clean and transformed speech improves held accuracy by 32 answers with the temporal features frozen. Its covariance penalty suppresses class-visible nuisance variation. The right panel compares the heads on the same features; the left shows all subsequent unrestricted encoder passes. The selected trained prefix reaches 518/657 (78.84%) versus 510/657 (77.63%) for the combined model on the reused disjoint audit. Audit labels do not choose the checkpoint.

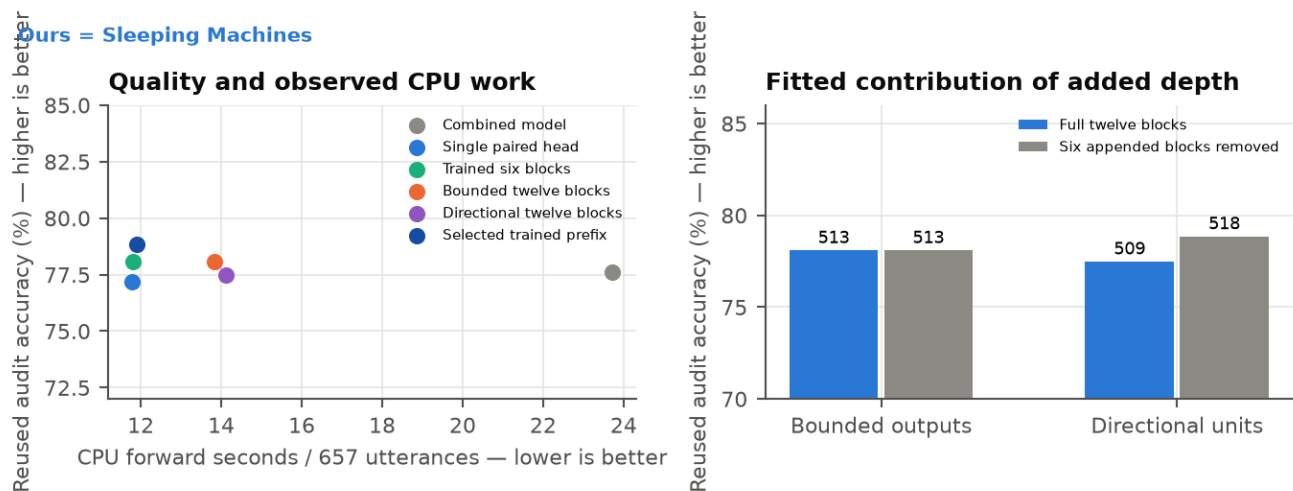
Training the directional twelve-block extension, then selecting its six-block prefix on development, retains the combined model's 408 correct answers with lower NLL and 395,814 deployed parameters. The extra training blocks are removed after their learned contributions reduce held accuracy. This improves deployment quality/work; it does not establish a positive deep-block accuracy gain.

Seed 6, private train-file speakers 3/6; official-test parity remains unmeasured. Head fitting uses 6,144 unique fitting utterances: one clean view for the first arm, clean plus one transformed view for the paired arm. The arms also change regularization and use three/two encoder passes respectively, so total budgets are not matched. All continuation epochs are plotted; selection uses development accuracy, then NLL. Extra teacher/cache/head work and inherited fitting must be charged. Modal/vector maps are locally dense; no empty ticks or event-pair attention are added. The selected prefix additionally inherits the full twelve-block fitting pass; pruning does not erase that training cost. Prefix/full choice is post-hoc private-development selection. Formulae and numerical checks: THEORY §§237-264. One CPU forward evaluation takes 11.90 s for the selected prefix versus 23.71 s for the combined model. Packing/query included, loading excluded; one timing observation, not joules.

Appendix A (continued). Making depth useful

Identity growth preserves the classifier and old teachers while added output maps receive label credit. They must also change useful features. A tightly bounded twelve-block extension learns weights but changes no audit decisions when its six appended blocks are removed.

Ours: one matched fitting pass	Parameters	Fit accuracy ↑	Private accuracy ↑	NLL ↓
Six-block control	395,814	90.72%	78.12%	0.639
Twelve blocks: bounded outputs	696,888	90.72%	78.12%	0.639
Twelve blocks: directional units	696,120	90.77%	78.32%	0.644



Directional conditioning normalizes temporal-state features before their output map. An invertible coordinate change rescales classifier-sensitive directions and preserves hidden null directions for later computation. The fixed transform folds into an ordinary map at deployment. Initial outputs and old teachers remain exact; fitting replays verify the actual proposed update.

The directional model reaches 509/657 on the reused audit. Removing its six appended blocks changes 11 predictions: 0 are correct only with the blocks and 9 only without them. This measures fitted contribution with the trained prefix/head retained; it is not a retrained architecture comparison.

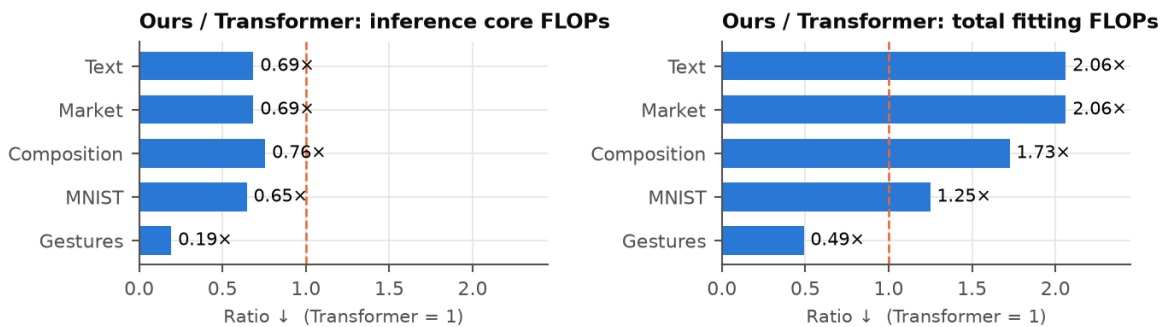
All arms inherit the paired-head checkpoint and use 6,144 fitting utterances, the same order, channel/time transformations and one encoder pass. Old-group LR is 0.0000203125; directional new groups use 0.0001953125 from fitting-only replay. Changed normalization and update coordinates form one intervention. New blocks initially add 36 ms latency; labels supervise completed untimed utterances. Audit reuse is explicit; official-test parity remains unmeasured. CPU points are one warmed observation per model, packing/query included and loading excluded; energy is unmeasured. Weight-coordinate folding, all source work and added depth must be charged during training. Local maps remain dense, with no empty ticks or event-pair attention. Theory §§249–264.

Appendix B. Breadth of the common implementation

These small development screens test one implementation across tasks. Text and market variants include separately fitted statistical evidence. Ours denotes Sleeping Machines; TF is the saved Transformer. Accuracy improves upward; prediction loss and FLOPs improve downward.

Task / quality direction	Ours: quality	TF: quality	Ours: inference MFLOPs ↓	TF: inference MFLOPs ↓
Text8 bpc ↓	2.915 bpc	3.729 bpc	1.26	1.84
Market event prediction nats/event ↓	3.823 nats/event	4.208 nats/event	1.26	1.84
Temporal composition accuracy ↑	97.3%	90.2%	0.29	0.38
MNIST accuracy ↑	75.8%	69.9%	1.66	2.57
Event-camera gestures accuracy ↑	59.1%	15.9%	23.99	124.23

Ours = Sleeping Machines



Ours uses eight layers and TF two, both width 32, with the same neural-fitting examples, encoding, objective and eight epochs. Inference counts maps/scans (ours) and maps/attention (TF): two FLOPs per multiply-add, excluding padding, scalar nonlinearities and expert preparation. Training includes backward, clipping, Adam and our evidence/calibration; its ledger follows.

Lower core inference work in every screen. Gestures use **5.18x less** with 59.1% versus 15.9% accuracy. That 44-query screen has an underfitting TF control; a general vision claim requires complete benchmarks and stronger references.

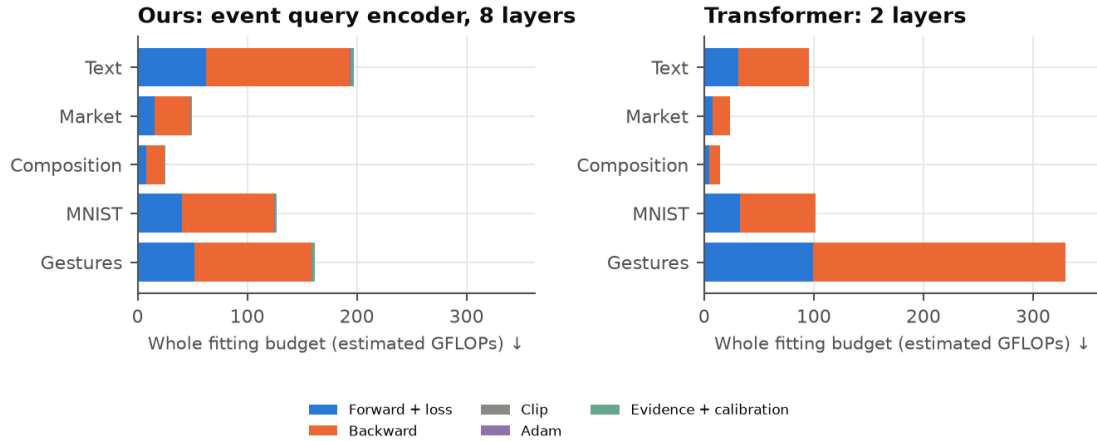
Why training can cost more: the race core evaluates all three candidate vector payloads during training, versus only the winner during inference. Its eight layers also exceed the reference's two. With short contexts, that work outweighs the saved attention cost; these rows do not show a training efficiency advantage. On the longer event-camera prefixes, the common model's estimated total uses 49.0% of the reference training arithmetic, including calibration.

Seed 6; neural fit/development counts: text 2,048/256, market 512/256, temporal 1,024/256, MNIST 1,024/256, gestures 88/44. The common text model also has a separately fitted 32,768-character evidence bank; market evidence is fitted on a prior day. The references have no such bank. MNIST uses pooled training-set images; gestures use first-second prefixes and disjoint users. Batch 16, or four for gestures. No official real-data test. Market fixed evidence: 3.670 nats/event.

Appendix B (continued). Total training cost

The ledger estimates the entire completed fitting budget for each reported model. It includes prediction and loss, backpropagation, gradient clipping and Adam across all eight epochs. The common model also pays for its evidence bank and initial readout calibration. These models were trained from initialization; there is no inherited neural fitting to omit.

Ours = Sleeping Machines



Model/task	Forward + loss	Backward	Clip	Adam	Evidence + calibration	Total
Language: Ours	62.633	130.873	0.165	0.658	2.930	197.26
Language: TF	31.246	63.990	0.083	0.334	0	95.65
Market: Ours	15.657	32.716	0.041	0.164	0.733	49.31
Market: TF	7.811	15.996	0.020	0.081	0	23.91
Temporal: Ours	7.725	16.546	0.081	0.324	0.364	25.04
Temporal: TF	4.773	9.540	0.041	0.162	0	14.52
Mnist: Ours	40.394	84.376	0.083	0.333	1.886	127.07
Mnist: TF	33.026	68.489	0.049	0.197	0	101.76
Dvs: Ours	51.580	107.386	0.028	0.112	2.404	161.51
Dvs: TF	99.023	230.261	0.014	0.057	0	329.35

All table values are estimated GFLOPs for the whole fitting run, not per query. A multiply-add counts as two operations. Forward/loss and backward use the saved E172 four-query operator trace scaled by recorded map/scan work. Ours uses its logged candidate-map/scan counts. Reference padding is reconstructed from all fitting prefix lengths, the original shuffle seed and batch sizes, including fused attention products. Clipping and Adam are charged once per original step. Other arithmetic and the small calibration eigensolver are estimates.

Event-target arithmetic excludes padding and simulator dispatch/allocation; required candidate maps, losing-value teaching, scans and learning remain charged. References use the same FLOP convention. Evaluation, search, encoding, special functions, integer/index work, comparisons and memory traffic are outside these totals. These single-seed screens have different depths/quality and do not measure event hardware, matched-quality cost or energy. Ledger: `estimate_training_work.py`.

Appendix B (continued). Completed language references

The Transformer and LSTM benchmarks have already been run. Their completed result files remain in the repository and are reused as reference targets for the full learned-event benchmark. Lower bits per character (bpc) means better prediction.

Model	Fitting characters / passes	Test bpc ↓	Full training FLOPs ↓
Ours: learned event-state model (planned)	10M / four passes	Pending	Pending
LSTM; width 512, one recurrent layer	10M / six passes	1.799	432.59T
Transformer; width 256, four layers	10M / four passes	1.908	888.78T
LSTM; width 512, one recurrent layer	90M / 6 passes	1.661	3.89P

The 10M references score exactly the same 999,999 text8 targets in [95M,96M), with frozen validation-selected weights and cold initial context. The 90M LSTM uses the same test interval and its saved recurrent scoring protocol. All use the historical 27-character alphabet and 200,000-character validation selection; data budgets, capacities and fitting passes differ.

Training estimates include every fitting step, forward/loss, backpropagation, gradient clipping and Adam. Backward is approximated as twice forward; multiply-add counts as two FLOPs. G/T/P mean billion/trillion/quadrillion. Validation/test evaluation, memory traffic and runtime are outside these arithmetic totals.

Ours: learned-language benchmark status

The planned comparison uses 10M fitting characters, four passes, 200,000 validation characters and the same 1M test interval. Priority is the integrated sparse/timed architecture documented in the next appendix. Completed dense-carrier fits remain diagnostic controls; their quality does not establish sparse-model performance. Numerical contracts and progressively larger integrated development fits precede promotion. The aligned full-test result and its fitting work remain pending. The preserved 28,403-parameter pilot's 3.351 development bpc uses a smaller fitting budget and different split; it is not a comparable test result.

Earlier 1M-character references also remain saved: LSTM **2.179** and Transformer **2.367 test bpc**, each with twenty fitting passes. The separate count/copy baseline and the cross-task Transformer/retrieval LSTM comparisons remain in their labeled sections and Appendix B.

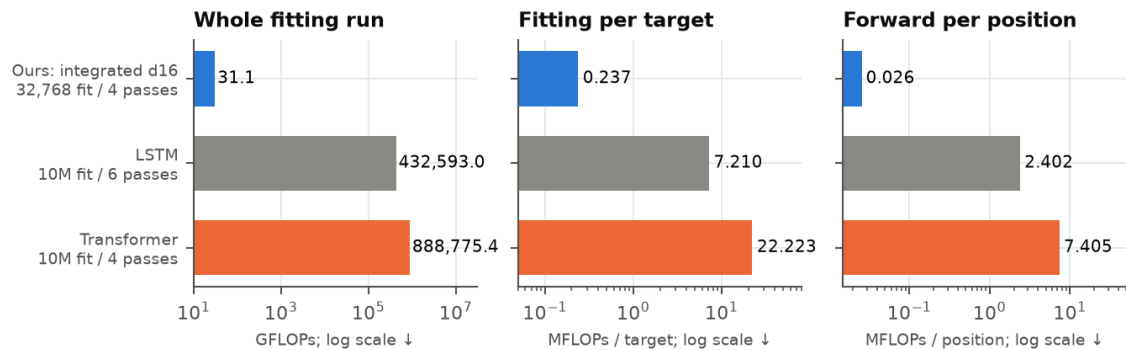
Saved evidence: 10M LSTM aligned result; 10M Transformer aligned result; 90M LSTM saved result. An earlier 90M Transformer attempt was interrupted by its RSS watchdog before producing a completed test result; its provenance is preserved.

Appendix B (continued). Ours: language work as scaling develops

This ledger updates from completed integrated-model stages. It shows the emerging work advantage alongside its quality and data budget. Per-target fitting work removes the difference in the number of presentations; it does not establish equal-quality superiority.

Ours = Sleeping Machines

Work estimates; data, model size and quality differ



Model	Fit / passes	bpc / split ↓	Whole fit GFLOPs ↓	Fitting MFLOPs / target ↓	Forward MFLOPs / position ↓
Ours / d16 / p2	32,768 / 4	3.121 / dev	31.053	0.237	0.026
Ours / d16 / p2	8,192 / 4	3.398 / dev	7.788	0.238	0.025
Ours / d16 / p4	8,192 / 4	3.426 / dev	14.721	0.449	0.032
LSTM / width 512	10M / six	1.799 / test	432,592.997	7.210	2.402
Transformer / width 256	10M / four	1.908 / test	888,775.443	22.223	7.405

Compare within a column: whole-fit totals use GFLOPs for every model; per-target and forward work use MFLOPs for every model. One GFLOP is 1,000 MFLOPs. Whole-fit totals also depend on the number of training presentations; the per-target column divides that out.

All table values, figures and ratios use arithmetic plus one operation per special function, matching the historical neural estimate convention. This is not a physical energy cost. Ours arithmetic-only whole-fit totals (GFLOPs): 32,768 / pool 2: 29.883; 8,192 / pool 2: 7.492; 8,192 / pool 4: 14.152. Separate special-function counts are preserved in each result.

The raw work gap is substantial. The completed 32,768-character integrated stage's representative forward estimate is **290x smaller** than the larger saved Transformer estimate; fitting work per target is **94x smaller**. These are configuration-level work ratios. Our development score and the reference official test score use different targets and data budgets. The gap is not a matched-quality supremacy claim.

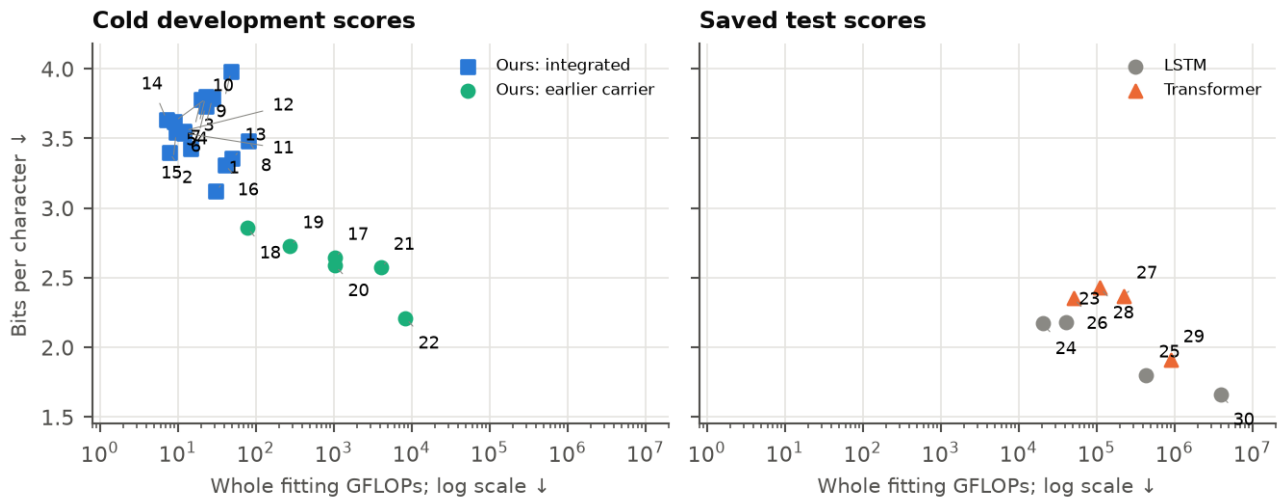
Ours: d denotes payload width and p pool size; fixed character pools and event depths. Each result records its validation interval and credit horizon. References: width-512 LSTM or four width-256 Transformer layers, 256-position fitting chunks and 999,999 aligned official test targets. Ours uses representative operator traces including counterfactual credit, backward, clipping and Adam; neural references use shape formulas and backward $\approx$ twice forward. RNG, indexing, memory traffic and evaluation passes are additional. Same-quality and iso-FLOP conclusions await comparable completed runs.

Appendix B (continued). Ours and neural controls: accuracy versus FLOPs

Each point is a completed model, not a projected scaling law. Left: ours on cold development characters, with integrated models and earlier carrier controls labelled separately. The new 2K screens score 2,047 development targets; the earlier ladders score 8,191. Right: saved neural test results. Lower bpc means better prediction; lower fitting work means fewer estimated operations. No curve is drawn between different model families or scoring splits.

Ours = Sleeping Machines

Quality versus fitting work; scoring protocols and data budgets differ



Model type	Fitting budget	bpc / split ↓	Whole fit GFLOPs ↓	Fitting MFLOPs / target ↓
Ours: integrated d16/p2	32,768 / 4 passes	3.121 / dev	31.053	0.237
Ours: carrier w128g	1,048,576 / 4 passes	2.210 / dev	8,373.302	1.996
LSTM: 512	90,000,000 / 6 passes	1.661 / test	3,893,396.042	7.210
Transformer: 256x4	10,000,000 / 4 passes	1.908 / test	888,775.443	22.223

The table selects the largest fitting budget currently completed for each family; the best score breaks ties. Point numbers refer to the following variant ledger, which lists all plotted variants. Variant labels: I = ours integrated payload/pool/data; IKV adds per-position race memory (S uses the content index); C = ours carrier width/data (g means content gates); L = LSTM width/data; T = Transformer width x layers/data; s denotes seed. K is 1,024 characters in ours labels; M is decimal million in neural labels.

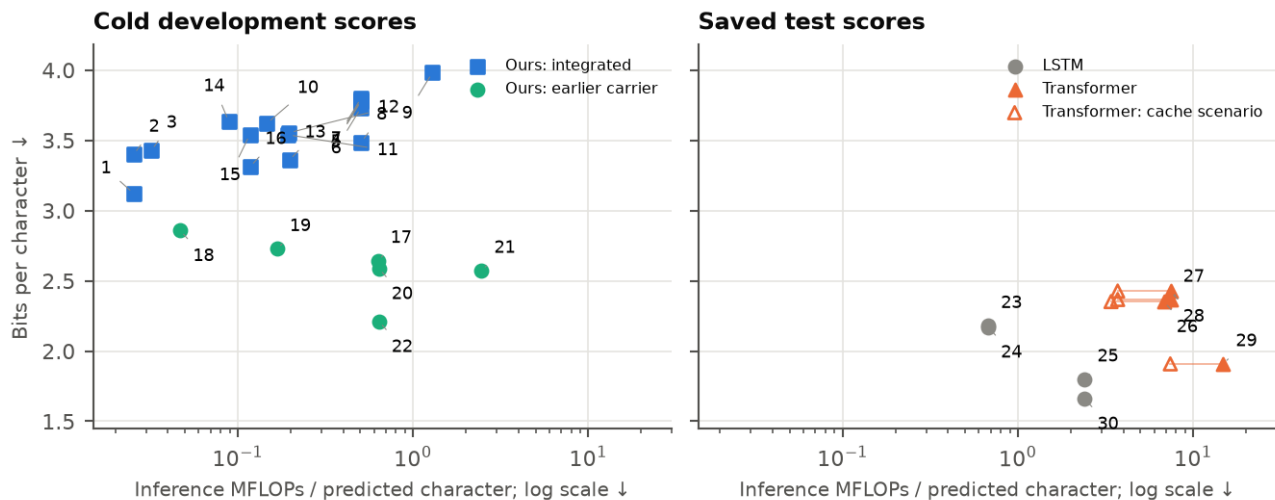
Estimates include learning, clipping and Adam, with unit-weight special functions. Ours uses representative operator traces; neural controls use shape formulas and backward approximately twice forward. Scoring splits, data, passes, capacity and credit differ; these panels are evidence inventories, not an iso-FLOP or equal-quality benchmark.

Appendix B (continued). Accuracy versus inference FLOPs

Inference predicts with frozen weights: no backward pass, clipping or optimizer update. These are the same completed checkpoints, quality scores and point IDs as the fitting graph. Ours uses saved forward operator traces; the integrated models read only winning values. LSTM and Transformer costs use shape estimates. Development and test evidence remain separate.

Ours = Sleeping Machines

Quality versus inference work; estimates, scoring protocols and data differ



Model type	bpc / split ↓	Inference MFLOPs / character ↓	Cost boundary
Ours: integrated d16/p2	3.121 / dev	0.0255	Winner-only inference trace
Ours: carrier w128g	2.210 / dev	0.6389	Saved forward operator trace
LSTM: 512	1.661 / test	2.4024	Recurrent shape estimate
Transformer: 256x4	1.908 / test	14.8104	Overlapping-window shape estimate

Solid Transformer points estimate its saved 256-position scorer: full windows advanced by 128 positions, approximately two forward positions per scored character (boundary/tail overhead omitted). Hollow points show a hypothetical one-step decode with cached keys/values and 256 available positions, using $L(24d^2 + 4Td + 30d + 20T) + 54d + 135$ unit-weight operations, $T = 256$. No cached decoder was run. Its plotted bpc belongs to the saved window scorer; learned positions reset between windows, so cache reuse has not been shown to preserve those scores.

Two FLOPs per multiply-add; special functions count as one operation. Ours traces include numerical clocks and loss scoring; neural elementwise overhead is approximate. Traces are representative warm-state costs, not full-stream measurements; growing KV candidate occupancy can change work. The separate KV pages also show projected event-architecture costs that remove numerical clock simulation. RNG, indexing, memory traffic and physical race energy are additional. These are work estimates, not latency or joules, and differing data, quality and evaluation protocols prevent a supremacy conclusion.

Appendix B (continued). Completed language variants and work

Variant	Params K	Fit / passes	bpc / split ↓	Whole fit GFLOPs ↓	Fit MFLOPs / target ↓	Inference MFLOPs / char ↓
1. Ours: l16/p2/32K/s6	361.4	32,768 / 4	3.121 / dev	31.053	0.237	0.0255
2. Ours: l16/p2/8K/s6	361.4	8,192 / 4	3.398 / dev	7.788	0.238	0.0255
3. Ours: l16/p4/8K/s6	720.0	8,192 / 4	3.426 / dev	14.721	0.449	0.0321
4. Ours: IHR2x32D8/2K/s6	3,819.5	2,048 / 4	3.786 / dev	27.731	3.387	0.5059
5. Ours: IHR2x32/u128@0.004D8/2K/s6	3,819.5	2,048 / 4	3.779 / dev	19.999	2.442	0.5058
6. Ours: IHR2x32/u64@0.002D8/2K/s6	3,819.5	2,048 / 4	3.733 / dev	22.753	2.779	0.5059
7. Ours: IHR2x32/u64@0.004D8/2K/s6	3,819.5	2,048 / 4	3.800 / dev	22.750	2.779	0.5060
8. Ours: IHR2x32/u128@0.004D8/8K/s6	3,819.5	8,192 / 4	3.485 / dev	79.953	2.440	0.5059
9. Ours: IHR4x32/u128@0.004D8/2K/s6	7,778.3	2,048 / 4	3.981 / dev	48.431	5.915	1.2898
10. Ours: IKV32D6/2K/s6	1,413.6	2,048 / 4	3.620 / dev	9.035	1.103	0.1466

Each row retains its original architecture, fitting budget and score. The selected 10M LSTM/Transformer rows use the aligned 999,999-target scores; other neural rows retain their original E64 test scorers. The 90M LSTM uses its saved recurrent scoring protocol. Carrier and integrated development scores use frozen evaluation; integrated official scores appear only after their full test completes. Validation/test work, RNG and physical traffic are outside fitting totals. Sources: E64/E174, saved AWS E64 results and the completed parallel_language and episodic_language JSON records. The global ledger uses emulator floating arithmetic consistently; fitting work per target divides by actual training target presentations. The separate KV page reports architectural projections. No new dense model was trained.

Appendix B (continued). Completed language variants and work

Variant	Params K	Fit / passes	bpc / split ↓	Whole fit GFLOPs ↓	Fit MFLOPs / target ↓	Inference MFLOPs / char ↓
11. Ours: IKV32D8/2K/s6	1,883.9	2,048 / 4	3.539 / dev	11.992	1.465	0.1941
12. Ours: IKVS32D8/2K/s6	1,883.9	2,048 / 4	3.554 / dev	11.999	1.465	0.1968
13. Ours: IKVS32D8/8K/s6	1,883.9	8,192 / 4	3.357 / dev	50.006	1.526	0.1969
14. Ours: I32D6/2K/s6	1,388.9	2,048 / 4	3.633 / dev	7.207	0.880	0.0890
15. Ours: I32D8/2K/s6	1,850.9	2,048 / 4	3.542 / dev	9.557	1.167	0.1173
16. Ours: I32D8/8K/s6	1,850.9	8,192 / 4	3.311 / dev	40.243	1.228	0.1173
17. Ours: C128/128K	308.0	131,072 / 4	2.643 / dev	1,032.197	1.969	0.6296
18. Ours: C32/128K	21.7	131,072 / 4	2.858 / dev	77.197	0.147	0.0470
19. Ours: C64/128K	80.3	131,072 / 4	2.727 / dev	274.735	0.524	0.1675
20. Ours: C128g/128K	309.6	131,072 / 4	2.587 / dev	1,046.656	1.996	0.6389

Each row retains its original architecture, fitting budget and score. The selected 10M LSTM/Transformer rows use the aligned 999,999-target scores; other neural rows retain their original E64 test scorers. The 90M LSTM uses its saved recurrent scoring protocol. Carrier and integrated development scores use frozen evaluation; integrated official scores appear only after their full test completes. Validation/test work, RNG and physical traffic are outside fitting totals. Sources: E64/E174, saved AWS E64 results and the completed parallel_language and episodic_language JSON records. The global ledger uses emulator floating arithmetic consistently; fitting work per target divides by actual training target presentations. The separate KV page reports architectural projections. No new dense model was trained.

Appendix B (continued). Completed language variants and work

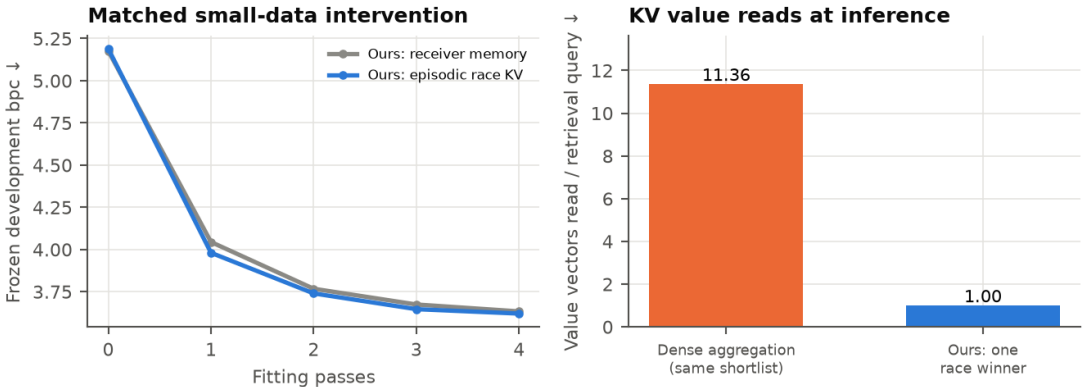
Variant	Params K	Fit / passes	bpc / split ↓	Whole fit GFLOPs ↓	Fit MFLOPs / target ↓	Inference MFLOPs / char ↓
21. Ours: C256g/128K	1,208.9	131,072 / 4	2.572 / dev	4,025.494	7.678	2.4571
22. Ours: C128g/1024K	309.6	1,048,576 / 4	2.210 / dev	8,373.302	1.996	0.6389
23. L256/1M	338.4	1,000,000 / 20	2.179 / test	40,628.875	2.032	0.6770
24. L256/10M	338.4	10,000,000 / 1	2.171 / test	20,306.115	2.032	0.6770
25. L512/10M	1,199.3	10,000,000 / 6	1.799 / test	432,592.997	7.210	2.4024
26. T112x8/1M	1,250.6	1,000,000 / 5	2.352 / test	51,107.144	10.223	6.7999
27. T256x2/1M	1,658.9	1,000,000 / 20	2.367 / test	222,614.402	11.133	7.4192
28. T256x2/10M	1,658.9	10,000,000 / 1	2.427 / test	111,261.602	11.133	7.4192
29. T256x4/10M	3,238.4	10,000,000 / 4	1.908 / test	888,775.443	22.223	14.8104
30. L512/90M	1,199.3	90,000,000 / 6	1.661 / test	3,893,396.042	7.210	2.4024

Each row retains its original architecture, fitting budget and score. The selected 10M LSTM/Transformer rows use the aligned 999,999-target scores; other neural rows retain their original E64 test scorers. The 90M LSTM uses its saved recurrent scoring protocol. Carrier and integrated development scores use frozen evaluation; integrated official scores appear only after their full test completes. Validation/test work, RNG and physical traffic are outside fitting totals. Sources: E64/E174, saved AWS E64 results and the completed parallel_language and episodic_language JSON records. The global ledger uses emulator floating arithmetic consistently; fitting work per target divides by actual training target presentations. The separate KV page reports architectural projections. No new dense model was trained.

Appendix B (continued). Ours: per-position race KV memory

Both integrated models fit 2,048 characters for 4 passes, with payload 32, 6 sparse receiver depths, seed 6 and 2,047 identical cold development targets. The KV arm retains separate historical keys and values at every depth; learned queries select one value through time. Incoming content is retained and gated with the retrieved message. No dense carrier is added.

Ours = Sleeping Machines



Ours: memory	Dev bpc ↓	Projected fit GFLOPs ↓	CPU fit GFLOPs ↓	Projected forward MFLOPs/char ↓
receiver	3.633	7.206	7.207	0.0890
kv	3.620	9.034	9.035	0.1464

Completed KV improvement over receiver memory: +0.013 bpc (positive is better). The index admits up to 8 recent matching-character entries plus 4 recent positions. Older entries outside these tails cannot be addressed by this index. Duplicates are removed. Development averages 11.36 keys scored and one value delivered per retrieval query. All 12,282 entries remain stored (3.00 MiB raw keys/values); the oldest selected entry is 1,829 characters old. This bounds reads, not stored history.

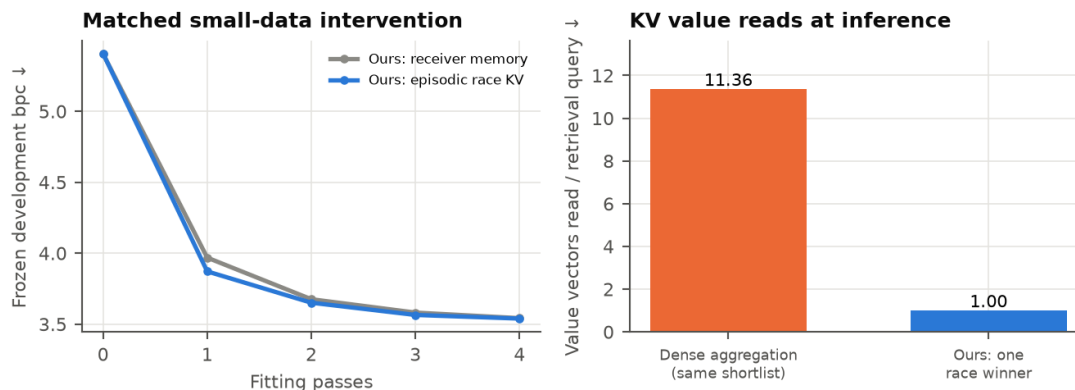
Physical clock competition replaces explicit numerical rate exponentiation and noise/rate division plus the bounded-delay simulation in the projected ledger. Query/key/value maps, scored candidates, gated content, backward, counterfactual teaching, clipping and actual Adam remain charged. Counts are representative first/mature/partial traces; special functions have unit weight here and are separate in JSON. Physical rate setting, clock circuits, index/address operations, RNG and traffic need their own implementation costs; FLOPs do not certify energy.

Temporal races avoid the explicit normalizing reduction/division and deliver one value at inference; training reads all admitted values for route credit. The orange bar is an analytical same-shortlist aggregation comparison, not another trained model. Candidate coverage is approximate and does not guarantee full-bank attention equivalence. Random-hyperplane indexing is an established primitive (Charikar, STOC 2002); novelty is not claimed for this index. Historical activations are detached at the credit boundary and are not recomputed after parameter updates. One seed and a small data budget; no equal-quality Transformer or frontier claim.

Appendix B (continued). Ours: per-position race KV memory

Both integrated models fit 2,048 characters for 4 passes, with payload 32, 8 sparse receiver depths, seed 6 and 2,047 identical cold development targets. The KV arm retains separate historical keys and values at every depth; learned queries select one value through time. Incoming content is retained and gated with the retrieved message. No dense carrier is added.

Ours = Sleeping Machines



Ours: memory	Dev bpc ↓	Projected fit GFLOPs ↓	CPU fit GFLOPs ↓	Projected forward MFLOPs/char ↓
receiver	3.542	9.556	9.557	0.1172
kv	3.539	11.990	11.992	0.1939

Completed KV improvement over receiver memory: +0.003 bpc (positive is better). The index admits up to 8 recent matching-character entries plus 4 recent positions. Older entries outside these tails cannot be addressed by this index. Duplicates are removed. Development averages 11.36 keys scored and one value delivered per retrieval query. All 16,376 entries remain stored (4.00 MiB raw keys/values); the oldest selected entry is 1,829 characters old. This bounds reads, not stored history.

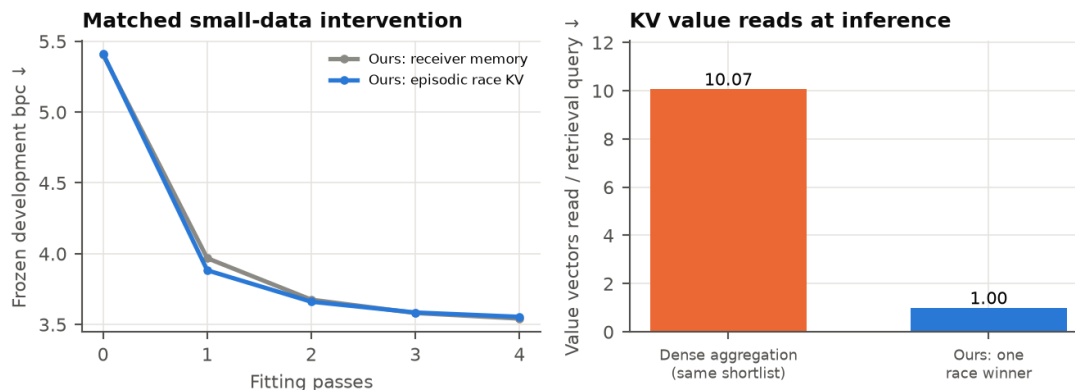
Physical clock competition replaces explicit numerical rate exponentiation and noise/rate division plus the bounded-delay simulation in the projected ledger. Query/key/value maps, scored candidates, gated content, backward, counterfactual teaching, clipping and actual Adam remain charged. Counts are representative first/mature/partial traces; special functions have unit weight here and are separate in JSON. Physical rate setting, clock circuits, index/address operations, RNG and traffic need their own implementation costs; FLOPs do not certify energy.

Temporal races avoid the explicit normalizing reduction/division and deliver one value at inference; training reads all admitted values for route credit. The orange bar is an analytical same-shortlist aggregation comparison, not another trained model. Candidate coverage is approximate and does not guarantee full-bank attention equivalence. Random-hyperplane indexing is an established primitive (Charikar, STOC 2002); novelty is not claimed for this index. Historical activations are detached at the credit boundary and are not recomputed after parameter updates. One seed and a small data budget; no equal-quality Transformer or frontier claim.

Appendix B (continued). Ours: per-position race KV memory

Both integrated models fit 2,048 characters for 4 passes, with payload 32, 8 sparse receiver depths, seed 6 and 2,047 identical cold development targets. The KV arm retains separate historical keys and values at every depth; learned queries select one value through time. Incoming content is retained and gated with the retrieved message. No dense carrier is added.

Ours = Sleeping Machines



Ours: memory	Dev bpc ↓	Projected fit GFLOPs ↓	CPU fit GFLOPs ↓	Projected forward MFLOPs/char ↓
receiver	3.542	9.556	9.557	0.1172
kv	3.554	11.997	11.999	0.1965

Completed KV improvement over receiver memory: -0.012 bpc (positive is better). The content index uses three random-hyperplane bits of learned keys/queries, with up to 8 recent/full-history samples in the query bucket and its one-bit neighbors, plus 4 recent positions. Duplicates are removed. Development averages 10.07 keys scored and one value delivered per retrieval query. All 16,376 entries remain stored (4.00 MiB raw keys/values); the oldest selected entry is 2,026 characters old. This bounds reads, not stored history.

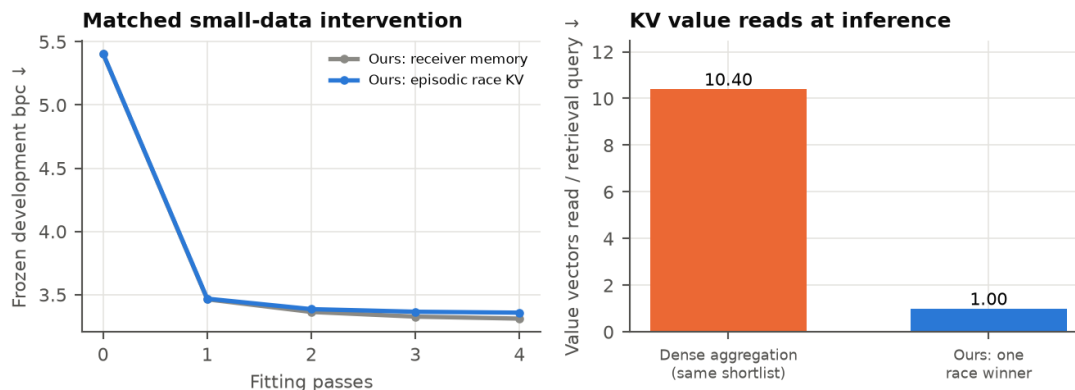
Physical clock competition replaces explicit numerical rate exponentiation and noise/rate division plus the bounded-delay simulation in the projected ledger. Query/key/value maps, scored candidates, gated content, backward, counterfactual teaching, clipping and actual Adam remain charged. Counts are representative first/mature/partial traces; special functions have unit weight here and are separate in JSON. Physical rate setting, clock circuits, index/address operations, RNG and traffic need their own implementation costs; FLOPs do not certify energy.

Temporal races avoid the explicit normalizing reduction/division and deliver one value at inference; training reads all admitted values for route credit. The orange bar is an analytical same-shortlist aggregation comparison, not another trained model. Candidate coverage is approximate and does not guarantee full-bank attention equivalence. Random-hyperplane indexing is an established primitive (Charikar, STOC 2002); novelty is not claimed for this index. Historical activations are detached at the credit boundary and are not recomputed after parameter updates. One seed and a small data budget; no equal-quality Transformer or frontier claim.

Appendix B (continued). Ours: per-position race KV memory

Both integrated models fit 8,192 characters for 4 passes, with payload 32, 8 sparse receiver depths, seed 6 and 8,191 identical cold development targets. The KV arm retains separate historical keys and values at every depth; learned queries select one value through time. Incoming content is retained and gated with the retrieved message. No dense carrier is added.

Ours = Sleeping Machines



Ours: memory	Dev bpc ↓	Projected fit GFLOPs ↓	CPU fit GFLOPs ↓	Projected forward MFLOPs/char ↓
receiver	3.311	40.241	40.243	0.1173
kv	3.357	49.998	50.006	0.1967

Completed KV improvement over receiver memory: -0.047 bpc (positive is better). The content index uses three random-hyperplane bits of learned keys/queries, with up to 8 recent/full-history samples in the query bucket and its one-bit neighbors, plus 4 recent positions. Duplicates are removed. Development averages 10.40 keys scored and one value delivered per retrieval query. All 65,528 entries remain stored (16.00 MiB raw keys/values); the oldest selected entry is 8,135 characters old. This bounds reads, not stored history.

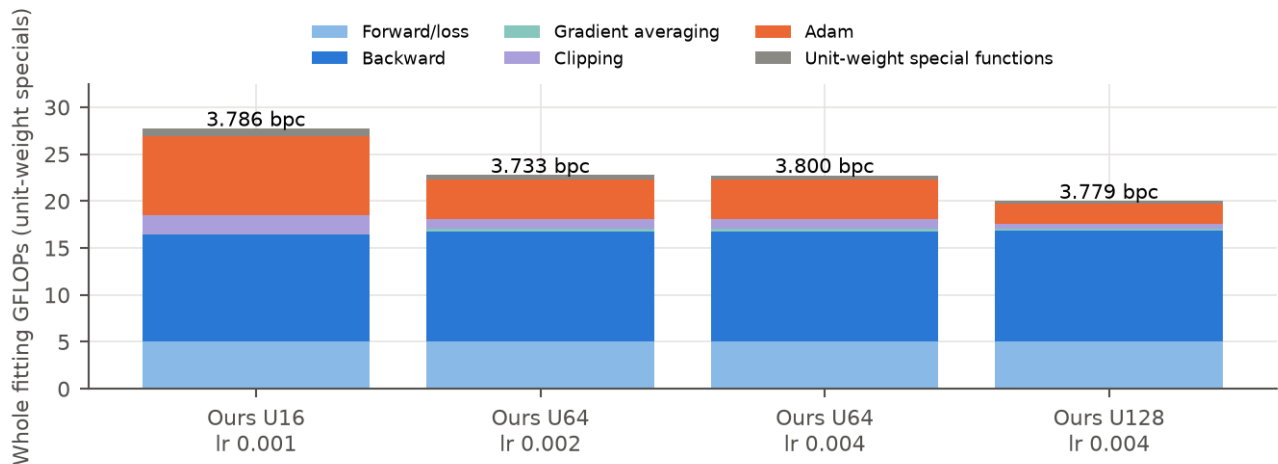
Physical clock competition replaces explicit numerical rate exponentiation and noise/rate division plus the bounded-delay simulation in the projected ledger. Query/key/value maps, scored candidates, gated content, backward, counterfactual teaching, clipping and actual Adam remain charged. Counts are representative first/mature/partial traces; special functions have unit weight here and are separate in JSON. Physical rate setting, clock circuits, index/address operations, RNG and traffic need their own implementation costs; FLOPs do not certify energy.

Temporal races avoid the explicit normalizing reduction/division and deliver one value at inference; training reads all admitted values for route credit. The orange bar is an analytical same-shortlist aggregation comparison, not another trained model. Candidate coverage is approximate and does not guarantee full-bank attention equivalence. Random-hyperplane indexing is an established primitive (Charikar, STOC 2002); novelty is not claimed for this index. Historical activations are detached at the credit boundary and are not recomputed after parameter updates. One seed and a small data budget; no equal-quality Transformer or frontier claim.

Appendix B (continued). Ours: cheaper learning updates

Credit still propagates over 16-character segments. Gradients are summed over U targets, normalized by their actual count, clipped once and used for one Adam update. All models here use two independent heads, payload 32/head, eight blocks, seed 6, four passes over 2,048 fitting characters and 8,191 frozen development targets.

Ours = Sleeping Machines



Ours: U / learning rate	Dev bpc ↓	Whole fit GFLOPs ↓	Fit MFLOPs / target ↓	Adam + clip GFLOPs ↓
U16 / 0.001	3.786	27.731	3.387	10.521
U64 / 0.002	3.733	22.753	2.779	5.225
U64 / 0.004	3.800	22.750	2.779	5.225
U128 / 0.004	3.779	19.999	2.442	2.645

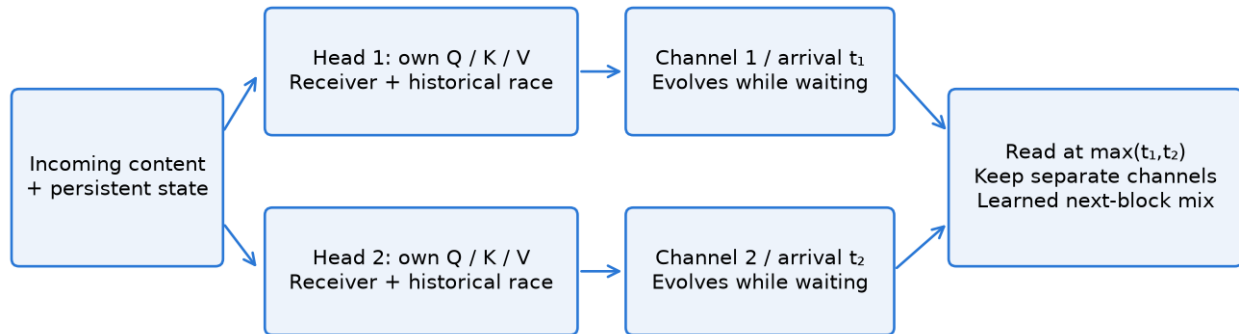
The cheapest schedule within the declared 0.05 bpc tolerance of the best pilot uses U128, lr 0.004: 3.779 bpc and 27.9% less whole fitting work than reference. The best quality is 3.733 bpc. These completed results support optimizer amortization in this configuration, not language-model supremacy.

Learning rates and warmup differ across configurations, so this is not an isolated optimizer-interval ablation. Selected checkpoints minimize frozen development loss over the fixed four passes; all fitting work remains charged. Each result is one seed. Candidate scoring, losing-value credit, backward and gradient accumulation/normalization remain in the operator ledger. Larger-data and repeat-seed comparisons must establish transfer of the selected schedule.

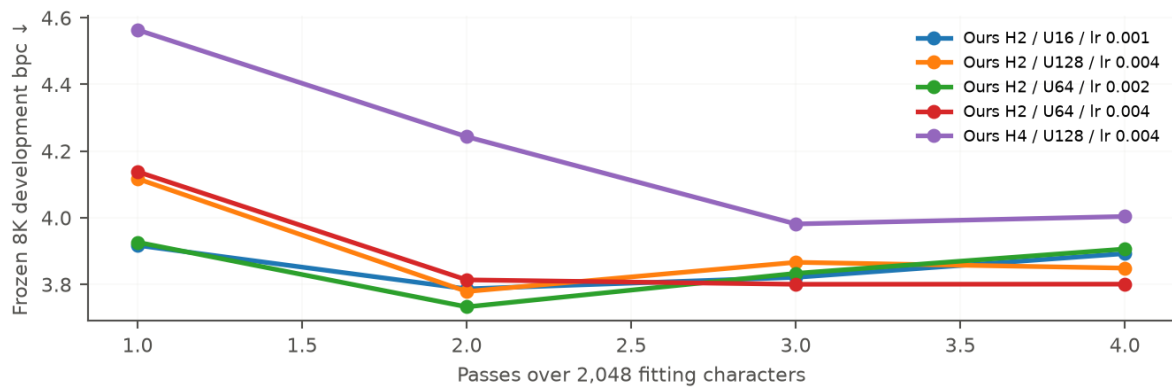
Appendix B (continued). Ours: independent temporal heads

Each head has its own receiver pool, historical bank and query/key/value/gate matrices. A winning content vector evolves through learned rotation and decay until its channel is read. The next block reads at the latest parallel arrival, preserves each channel and learns their mix. Heads need not arrive simultaneously. This is implemented in the CPU emulator; execution there is serial.

Ours = Sleeping Machines



Ours = Sleeping Machines



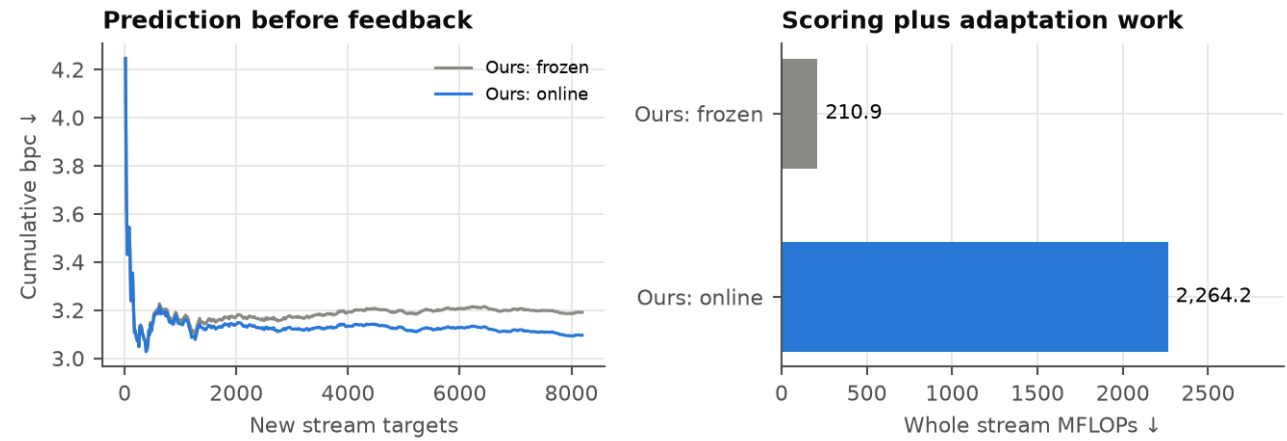
Ours: heads / update	Fit / passes	Dev bpc ↓	Whole fit GFLOPs ↓
H2 / U64 / lr 0.002	2,048 / 4	3.733	22.753
H2 / U64 / lr 0.004	2,048 / 4	3.800	22.750
H2 / U128 / lr 0.004	8,192 / 4	3.485	79.953
H4 / U128 / lr 0.004	2,048 / 4	3.981	48.431

Payload 32 per head: H2 total width 64, H4 total width 128; eight blocks. More heads also increase capacity, and source/channel dynamics differ from the old single-head model. The baseline H2 pilot selects epoch 2 and overfits later; no head-count quality benefit is established. Adam interval U is separate from 16-character credit. Training reads admitted losing values and charges gradients, clipping and optimizer work. Contracts pass for causality, independent projections, evolving channels, all-head gradients and exact next-update recovery. All completed variants remain in the ledger; this table shows the latest four records.

Appendix B (continued). Ours: separate online neural learning

Both arms start from the same selected integrated-model checkpoint and maintain persistent event memory on the same new development stream. The frozen arm retains its parameters. The online arm updates the complete neural backbone after making the causal predictions in each 16-character block, with no replay.

Ours = Sleeping Machines



Ours: mode	Stream bpc ↓	Whole stream MFLOPs ↓	Parameter change L2	Updates
Ours: frozen	3.191	210.903	0.000	0
Ours: online	3.096	2,264.163	1.551	512

The 8,191 targets come from [90,065,536, 90,073,728). Learning rate 0.0001 was fixed before this stream; Adam starts with fresh moments. Both arms use paired block race noise. Parameter updates precede only future blocks; the current target cannot change its own prediction. Their contexts can diverge after learning.

This measures block-delayed online adaptation, not instant per-character updates or a frozen official-test score. One inherited model/window/rate. All neural parameters may learn, including content, keys, clocks and retention; this differs from the earlier statistical expert-mixing-only ablation. Inherited fitting is additional and identical in both arms. Work estimates observe the actual first/middle/last blocks and include counterfactual learning, backward, clipping and Adam; RNG, indexing and physical traffic remain additional. Lower online loss, if observed, is evidence only for this protocol.

Appendix B (continued). Separate statistical language baseline

This count/copy predictor does not use the learned Sleeping Machines event backbone. It is a separate statistical system: order-0 through order-6 counts, backoff probabilities and a bounded causal copy cache, combined by learned mixing weights. Its quality/work results must not be attributed to the event architecture.

The statistical predictor and saved neural references score the same 999,999 character targets, starting from a cold context. Parameters are frozen during testing. Earlier observed test characters can supply causal context, including the mixture's bounded 256-character copy cache. Lower bits per character means better prediction.

Predictor	Test bpc ↓	Fitting and selection budget	Estimated training FLOPs ↓
Ours: separate statistical count/copy baseline	1.727	10M count fitting + three 1M mixing-rate trials	4.47G + integer count construction
Ours: count/copy plus causal word context	1.719	10M count fitting + three 1M mixing-rate trials	4.80G + integer count construction

The neural totals charge all original optimizer steps that produced the inherited E174 checkpoints: forward, estimated 2×-forward backward, clipping and Adam. Alignment evaluation is excluded. The statistical floating estimate charges expert probability preparation and all three mixing-rate trials, including local gradients and weight updates. Exp/log/root evaluations count as one operation in these language estimates. Count construction additionally uses about 80M integer count presentations, plus sorting, lookup and hashing; that work is not quantified as FLOPs. The floating totals alone cannot establish total compute, runtime or energy savings.

The count/copy mixture improves on LSTM by 0.072 bpc and Transformer by 0.181 bpc. These results establish useful specialized prediction; generic learned representations are assessed in the separate language screen.

The statistical baseline's count arrays occupy 66.55 MB. Vocabulary, capacities, optimization and fitting budgets differ from the neural references. The comparison does not measure total training energy or a matched-capacity advantage.

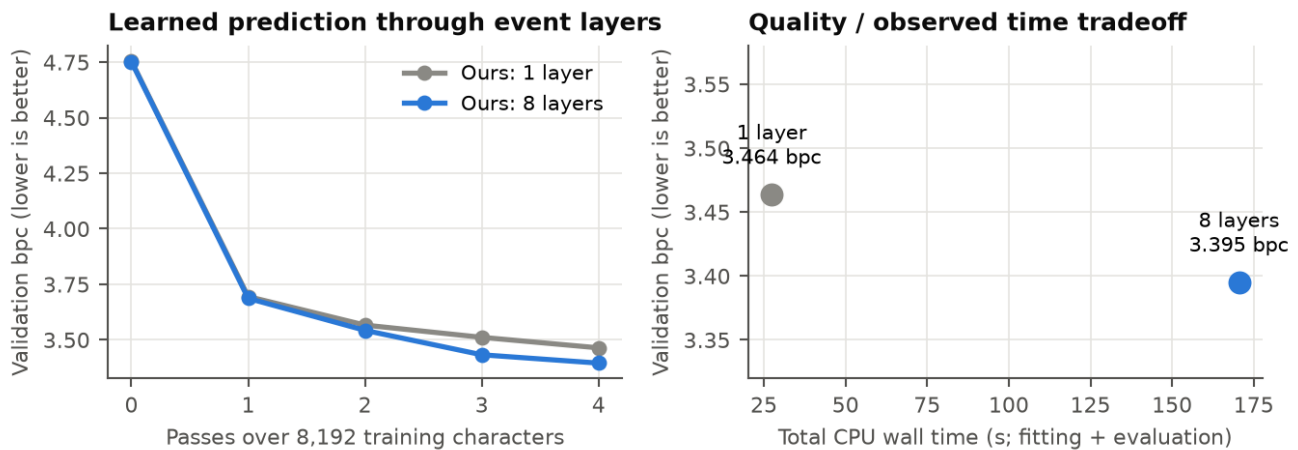
All three use the same historical 27-character alphabet. Modern shared subword tokenization is a separate comparison gate for the learned event architecture, described later in this appendix.

One exploratory seed. Text8 offsets: count fitting [0,10M), mixing-weight validation [90M,91M), test [95M,96M); test index zero is excluded for all three predictors. The mixture selects its update rate on validation. Saved neural weights are unchanged. Results: E173/E174; stream contract: E175. At 90M training characters, reference test scores are 1.661 for the LSTM. Estimated training work (forward, backward, Adam and gradient clipping): LSTM: 3.89 PFLOP. Shape-based estimates count multiply-add as two operations; backward is approximated as twice forward. Validation/test inference is excluded. four-layer Transformer reference results are pending. These are single-seed comparisons; capacities and fitting budgets are not matched.

Appendix B (continued). Learned language and depth

A bounded screen trains the common event backbone to predict the next character. Both configurations use width 32, the same 8,192 training characters, four passes, 32-character contexts and 1,024 validation predictions. All eight layers' value, route and memory-time parameters update. Lower bits per character means better prediction.

Ours = Sleeping Machines



Ours: depth	Learned parameters	Validation bpc: lower is better	Total CPU wall time
1	7,671	3.464	27.4 s
8	53,430	3.395	170.5 s

Eight layers improve validation loss from 4.752 to 3.395 bpc, versus 3.464 with one layer. Shuffling preceding characters while preserving the last character, count and timestamps increases the deeper model's loss to 3.805; replacing preceding context raises it to 3.777. These frozen input probes show context sensitivity, not a retrained baseline comparison.

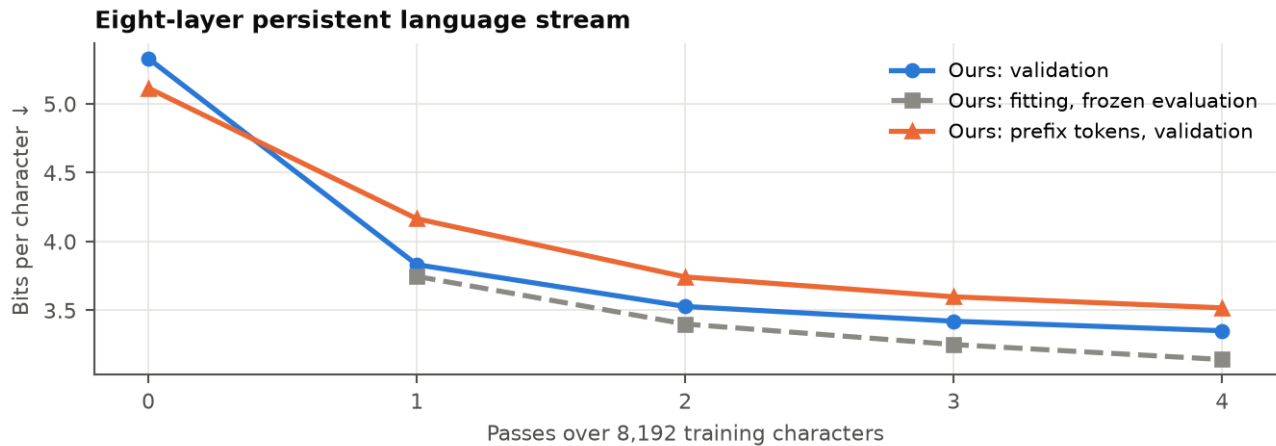
The deeper model has more parameters and takes more CPU time. The quality/time panel includes fitting and evaluation, with backpropagation and optimizer updates executed during fitting. These bounded-query models replay preceding context. The following persistent implementation consumes each character once. Physical memory traffic and joules remain unmeasured.

One seed; different parameter counts. This establishes a generic learned-language foothold and a small depth gain, not competitive large-scale representation, a matched tuned dense-model advantage or a scaling law. The statistical 10M-character mixture remains a separate result. Official test data are untouched. E133 preserves commands, source/data hashes, layer diagnostics and work coverage.

Appendix B (continued). Persistent learned language

The event-state language model consumes each character once and retains local modal memories and its delayed-message queue. Chunk boundaries truncate learning credit without discarding the observed history. Only actual event arrivals evaluate layers; text time is measured in token intervals.

Ours = Sleeping Machines



Ours: representation	Parameters	Fitting bpc ↓	Validation bpc ↓	Deliveries/pass
Characters	28,403	3.142	3.351	65,784
Causal prefix tokens	35,163	3.085	3.517	45,816

Validation loss falls from 5.329 to 3.351 bpc. All eight layer teachers are nonzero in every fitting pass. Each pass consumes 8,223 characters including warmup and makes 65,784 block deliveries. The result establishes learning with persistent causal state and no prefix replay.

A train-only 131-token prefix dictionary reduces layer deliveries by 30.4% and observed CPU time by 26.2%, while validation bpc is 3.517. Exact partial-token marginalization scores identical raw targets. Its larger vocabulary fits better but generalizes less well in this small screen: compression alone does not explain or resolve the quality gap.

One seed; 28,403 parameters, width 32, sixteen temporal modes per block. Four passes, 128 Adam steps/pass, credit truncated every 64 characters, 31 warm characters and 1,024 validation targets. Target offsets match the bounded E133 screen; topology, capacity, history and update counts differ, so this is not a matched intervention. Total CPU wall time 364.9 s including fitting/evaluation; peak RSS 364.8 MiB. These are event counts and observed resources, not total arithmetic, physical memory traffic or energy. No official test or large-corpus claim. E176.

Appendix B (continued). Learned event language: training and inference work

These counts belong to the learned eight-layer persistent Sleeping Machines event model that reaches 3.351 validation bpc. It has no count/copy/word experts. Primary counts describe the logical event algorithm; simulator dispatch/allocation is excluded.

Ours: fitting stage	Estimated arithmetic FLOPs
Forward and loss	1.96G
Backward	4.77G
Gradient clipping	43.52M
Optimizer	173.95M
Warmup	7.42M
Total	6.96G

The entire completed budget includes 32,768 fitting targets, 512 Adam/clipping steps and four stream warmups. Special functions add 85.431M evaluations, reported separately from arithmetic FLOPs.

Ours: inference boundary	Per character
Prediction plus NLL-scoring arithmetic	59,741 FLOPs
Additional special functions	1,212 evaluations

A 64-character saved-checkpoint trace measures forward/loss, backward, clipping and warm Adam; the fitting ledger scales those stages by the original 512 steps and separately charges warmup. This is a representative arithmetic estimate, not a whole-run trace. The traced chunk has complete floating-operator formula coverage. Index/queue work, comparisons, memory traffic and evaluation passes are outside the fitting arithmetic boundary.

The 10M-character LSTM/Transformer checkpoints have different data, capacity and achieved quality. Dividing their full fitting budgets by this small run would not establish a fair training advantage. Full learned-event runs must complete before a larger aligned quality/work comparison.

Evidence: [event_language_work/local_event_language_work_20260930T124646Z.json](#); quality: E176. Two arithmetic FLOPs per multiply-add. Additional unit-weight special functions would give a different logical-operation total; no event-device runtime or energy has been measured.

Appendix B (continued). Statistical online-learning pilot

The official language comparison freezes parameters during evaluation. A new development-only pilot asks whether causal local learning improves prediction: score each character first, reveal it, then update only the expert mixing weights. Count experts stay frozen and both arms use identical causal copy-cache behavior.

Ours: statistical expert set	Frozen bpc ↓	Online bpc ↓	Extra update FLOPs
Without word	2.605	2.564	2.75M
With causal word	2.537	2.487	3.21M

This pilot reuses a 100,000-character count checkpoint and its learning rate selected on an earlier 2,048-character validation window. It scores 8,191 targets in a fresh 8,192-character development window [90,032,768,90,040,960), excluding the first target. Each adaptive arm performs 8,192 updates, including first-position warmup. Official test data are untouched.

One checkpoint and one window; a specialized adaptive readout, not deep learned TTT or a frontier result. The extra arithmetic column counts only local gradient/weight updates, on top of shared prediction and inherited fitting costs. Total pilot CPU wall time is 0.587 s. Evidence: `online_language/local_online_language_20260930T121741Z.json`.

Compute allocation is the next architectural hypothesis

Independent budgets for active width/depth, dormant capacity, temporal memory, retrieval, credit and adaptation may let extra work buy more prediction quality. The new theory derives conditional marginal-value allocation and retrieval-error bounds, and identifies exposure, routing overhead and hardware utilization as possible limits. A small-scale win does not prove a better scaling exponent or a widening frontier advantage.

Next learned-language comparisons need shared modern subword tokenization, Unicode/byte coverage, suitable rotary/relative position, packed optimized Transformer kernels, sparse MoE and modern recurrent/attention-hybrid controls. Existing event memory already uses relative-time rotations. Token coordinates must remain distinct from learned scheduling delays.

Known sequences can be fitted with input-known affine scans and then generated sequentially with persistent state. Sequence-parallel fitting also permits Transformer adaptation between tokens/chunks. Compare frozen and online arms at identical observations and adaptation budgets; charge cache consistency and update work.

Theory §§280–286 and the frontier compute protocol specify these tests. Modern architecture and larger-scale comparison arms remain proposed, not completed.

Appendix C. Preserved historical results and revisions

Earlier result files remain part of the research record. The following numbers explain older report headlines and why their interpretation changed. They are preserved here with the identified protocol errors; they are not current valid benchmark comparisons.

Earlier statistical language results

Ours: fitting characters	Frozen historical bpc	Online historical bpc
1M	1.808	1.782
10M	1.613	1.593
90M	1.504	1.483

These E79 mixtures use counts, a partial-word expert and a 256-character copy window. The partial-word key depended on whether the target character was a space: changing the unseen target changed the predicted distribution. The old 1.613/1.504 headlines therefore cannot establish a causal language advantage. The corrected E173 10M results are 1.727 without word context and 1.719 with causal word context; a corrected 90M mixture comparison remains open.

Earlier event world-model results

Historical predictor	Day 6 log-likelihood ↑	Day 7 log-likelihood ↑
Ours: event hazard + rate/flow state	-2.184	-1.999
Ours: event hazard + per-type state	-2.159	-1.973
Ours: event hazard + per-type state; finer gap bank	-1.912	-1.734
Saved Transformer Hawkes reference	-1.971	-1.816

The event hazard models use sparse conditional memories and local rate/flow state. Their frozen test parameters and causal state updates are useful mechanisms. However, the event-size threshold was fitted across all seven pilot days, including the evaluation days, for both the event and neural references. Day resets and warmup exclusions also differ. The recorded gap needs fitting-only preprocessing and aligned rescoreing before it supports a held-day advantage.

Source and numerical review; E79 language records; E57 world-model records. Preserved timing, composition, retrieval and modular results remain in the main report. New learned-model benchmarks add evidence; they do not erase these earlier runs.

Appendix D. Evidence and metric definitions

Metric	Interpretation
Bits per character	Held-out negative log probability in base two; lower is better next-character prediction.
Accuracy	Fraction of correct class decisions on the declared development or test protocol.
Event likelihood	Scores both the next event type and waiting time, including the observed silence.
Arithmetic FLOPs	Multiply-add counts as two operations. Tables state whether special functions are separate or assigned unit cost; these conventions must be aligned before forming ratios.
Logical operations	The structured-task ledger assigns 2 units per MAC and 1 per other scalar arithmetic/nonlinear operation or estimated sort comparison.
Resource boundary	Event-target arithmetic includes required active algorithm work. Memory traffic, queues, indexing and simulator overhead are reported separately where measured. Activity counts alone do not determine total FLOPs.
Energy	Measured total joules over an explicit boundary. Operation estimates and CPU timings support work comparisons, but are not joule measurements.

The evidence is preserved in versioned result summaries with configurations, split identities, learning curves and source hashes. E173/E174 support the language comparison; E61 supports retrieval; E34/E53/E54 support native composition; E41 supports the original periodic computation. E121/E124 establish consolidated arithmetic and its certificate; E123 supplies the new dense controls and E124 the operation ledger. E118/E119/E122/E125/E126 support deep speech, readout and causal-context comparisons; E127–E131 audit credit geometry, hard race boundaries and separate key/value learning; E132 checks a joint race-credit formalism, E133 supplies the language/depth screen, and E134–E135 test whole-value credit and content-selective temporal memory. E136 audits reversible augmented transport and its supervised memory boundary, including twelve-layer query/learning interventions. E137 tests compact memory queries and class-visible credit geometry. E138–E141 examine richer source messages and trainable signed temporal memory, with exact local teacher and initial-nesting contracts. E142 establishes signed-state and first-coalescing identities; E143 tests a larger nonlinear temporal residual learner, and E144 audits simultaneous state/query pooling. E171 reproduces the consolidated screens and selected speech answers, and checks causal input boundaries. E172 records complete training-step arithmetic; E175 checks the generic persistent language stream.

The project theory index contains formal assumptions and proofs. Research findings retain detailed analyses and the full experimental record. The model documentation describes reproducible configurations and operational procedures. This report presents the project, its evidence and its potential.